



Università degli Studi di Milano Bicocca

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di laurea in Informatica

L'insieme di Mandelbrot su CPU e GPU

Analisi comparativa di OpenMP, MPI e CUDA

Elaborato a cura di:

Gaia La Rocca

Matricola: 866582

Elaborato per il Corso di Sistemi di Calcolo Parallelo

Anno Accademico 2025-2026

Indice

1	Introduzione	1
1.1	Ambiente sperimentale	1
2	Frattali	3
2.1	L'insieme di Mandelbrot	3
2.2	Pseudocodice	8
3	Metriche e Modelli di Prestazione	8
3.1	Metriche della baseline sequenziale	8
3.1.1	Tempo di esecuzione	9
3.1.2	Modello analitico del carico computazionale	9
3.1.3	Profilo di carico e indici di sbilanciamento	10
3.2	Metriche delle implementazioni parallele	11
3.2.1	Tempo parallelo	11
3.2.2	Speedup ed efficienza	11
3.2.3	Legge di Amdahl	12
3.2.4	Metrica di Karp–Flatt	12
3.3	Previsione delle prestazioni parallele dal profilo di carico seriale	12
3.4	Ottimizzazioni avanzate disponibili e scelta della baseline	13
3.5	Artefatto di riferimento e riproducibilità	13
4	Implementazione Seriale	13
4.1	Impatto della risoluzione e della soglia iterativa	14
4.2	Frammenti di codice rilevanti	14
4.3	Risultati e discussione	15
5	Parallelizzazione	18
5.1	Frazione parallelizzabile e scelte di decomposizione	18
5.2	Previsioni dal profilo di carico seriale	19
5.3	OpenMP: memoria condivisa	20
5.3.1	Design dello sweep	20
5.3.2	Metriche specifiche	21
5.3.3	Frammenti di codice rilevanti	21
5.3.4	Analisi dei risultati	22
5.4	MPI: memoria distribuita	26
5.4.1	Design dello sweep	27
5.4.2	Metriche specifiche	28
5.4.3	Frammenti di codice rilevanti	28
5.4.4	Analisi dei risultati	30
5.5	Validazione a posteriori delle previsioni del baseline	34
5.6	CUDA: acceleratore grafico	36
5.6.1	Design dello sweep	37
5.6.2	Metriche specifiche	39
5.6.3	Frammenti di codice rilevanti	40
5.6.4	Previsioni dalla matrice degli escape time	42
5.6.5	Analisi dei risultati	43
6	Conclusioni	48
6.1	Il problema: un'irregolarità misurabile prima di parallelizzare	48
6.2	CPU: la regola di assegnamento decide lo speedup	49

6.3	GPU: la stessa irregolarità pesa poco sul tempo	49
6.4	Le ipotesi iniziali alla prova dei dati	50
6.5	Risultati inattesi	50
6.6	Limiti e sviluppi	51

Bibliografia	51
---------------------	-----------

1 Introduzione

Il presente elaborato si propone di condurre un'analisi comparativa delle prestazioni ottenibili parallelizzando un medesimo problema computazionale mediante i tre paradigmi affrontati nel corso: la programmazione a memoria condivisa con OpenMP, la programmazione a memoria distribuita con MPI e la programmazione su acceleratori grafici con CUDA. L'obiettivo non è ridurre il tempo di esecuzione né competere con le implementazioni di riferimento, bensì misurare e interpretare in modo rigoroso il comportamento di ciascun paradigma, mettendo in relazione i risultati sperimentali con i modelli analitici del calcolo parallelo.

Come caso di studio è stato scelto il calcolo dell'*insieme di Mandelbrot*. La scelta non è casuale. Il valore di ciascun pixel dipende esclusivamente dalle proprie coordinate, e il problema è quindi *imbarazzantemente parallelo* (embarrassingly parallel): i pixel possono essere calcolati in totale indipendenza, senza che le unità di calcolo debbano scambiarsi dati. Questa apparente semplicità nasconde tuttavia una difficoltà tutt'altro che trascurabile: il costo del singolo pixel dipende dal numero di iterazioni necessarie a determinarne l'appartenenza all'insieme. I punti interni raggiungono il limite massimo di iterazioni, mentre quelli esterni divergono, e quindi terminano, per lo più dopo pochi passi. La distribuzione del carico che ne risulta è fortemente disomogenea, e non è nota prima di aver calcolato l'immagine almeno una volta.

È questa irregolarità a costituire il nucleo analitico dell'elaborato. La tesi che attraversa l'intero lavoro è che un'unica causa, la variabilità del numero di iterazioni per pixel, si manifesti in forme diverse a seconda dell'architettura. Sulla CPU si presenta come sbilanciamento del carico (load imbalance) fra i thread o i processi che si spartiscono l'immagine. Sulla GPU invece si presenta come divergenza dei warp (warp divergence): i thread eseguiti insieme in uno stesso gruppo, il warp, terminano solo quando termina il più lento. Riconoscere questa radice comune permette di trattare i tre esperimenti non come esercizi scollegati, ma come approcci alternativi a un medesimo problema.

Le prestazioni di ciascun paradigma sono misurate con metriche consolidate: speedup ed efficienza, affiancati dalle leggi di scalabilità e da indici di sbilanciamento ricavati dal profilo di carico seriale. La legge di Amdahl fornisce il limite allo speedup imposto dalla frazione seriale del programma, mentre la metrica di Karp–Flatt, osservata al crescere del grado di parallelismo, aiuta a distinguere se la perdita di efficienza derivi da una componente seriale oppure dallo sbilanciamento del carico e dalla comunicazione.

Proprio perché privo di comunicazione, tuttavia, il calcolo dell'insieme di Mandelbrot non sollecita il regime in cui le prestazioni sono dominate dallo scambio e dalla sincronizzazione dei dati. Un complemento naturale sarebbe il Buddhabrot, che condivide la stessa iterazione. Anziché però scrivere un solo pixel per punto, il Buddhabrot incrementa lungo l'orbita di ciascun punto divergente numerose celle di un istogramma condiviso, introducendo contesa sulla memoria. Non è stato affrontato in questo lavoro, e viene ripreso fra gli sviluppi (sezione 6).

Quanto ai confini del lavoro, l'elaborato non offre una trattazione della geometria frattale né dei fondamenti matematici dell'insieme: la sezione 2 li richiama solo nella misura in cui spiegano l'origine dell'irregolarità del carico. Sul piano sperimentale, le misure dipendono dalle risorse di calcolo disponibili, descritte nella sezione 1.1. I valori assoluti contano quindi meno degli andamenti e del confronto fra paradigmi, e particolare cura è dedicata alla metodologia di misura e alla riproducibilità dei risultati.

1.1 Ambiente sperimentale

Tutte le misure sono state eseguite come job batch su un cluster HPC gestito dallo scheduler SLURM (versione 23.11.6), con sistema operativo Rocky Linux 8.9. Si riportano di

seguito le caratteristiche dell'ambiente che incidono sulla progettazione degli esperimenti e sull'interpretazione dei risultati.

Nodo di calcolo. L'utenza impiegata è abilitata a tre partizioni (`ulow`, `only-one-gpu` e `ice4hpc`), tutte associate al medesimo nodo, `gnode01`; le campagne CPU sono eseguite su `ulow`, quelle GPU su `only-one-gpu`. Il nodo monta due processori AMD EPYC 9354 (architettura Zen 4) da 32 core ciascuno, con estensioni AVX2 e AVX-512. Il `simultaneous multithreading` è disattivato, per cui i 64 core visibili al sistema coincidono con i core fisici. Ogni socket costituisce un nodo NUMA con circa 377 GiB di memoria, per un totale di 755 GiB; il sistema riporta per l'accesso alla memoria dell'altro socket un costo relativo di 32, contro 10 per la memoria locale.

Da questa configurazione discendono tre conseguenze. Nessuna esecuzione può estendersi su più nodi, per cui anche MPI opera su memoria condivisa (sezione 5.4). Il tempo seriale T_1 e i tempi paralleli $T_{||}(p)$ sono misurati per costruzione sullo stesso hardware. Il nodo è infine condiviso con i job di altri utenti: la partizione non prevede `preemption` né condivisione dei core allocati, ma banda di memoria e cache di ultimo livello restano contese, ed è per questa ragione che ogni configurazione viene ripetuta e se ne riporta il tempo minimo.

Acceleratore grafico. Il nodo ospita otto schede NVIDIA L40S, ma la partizione dedicata alla GPU ne assegna una sola per job: tutte le misure CUDA si riferiscono pertanto a una singola scheda. La scheda ha architettura Ada Lovelace, `compute capability` 8.9 e 46 068 MiB di memoria rilevati da `nvidia-smi`; opera con il driver 545.23.08 ed è collegata all'host tramite un link PCIe 4.0 x16. L'accesso ai performance counter della GPU è riservato agli amministratori, e le misure con il profiler `ncu` restituiscono l'errore `ERR_NVGPUCTRPERM`: la divergenza dei warp, che richiederebbe un profiler, è quindi stimata con un proxy deterministico (sezione 5.6.2).

Software. Il compilatore è GCC 8.5.0, fornito dal sistema; MPI è OpenMPI 4.1.6 e CUDA è il toolkit 12.3.2, mantenuto invariato per l'intera campagna, poiché un toolkit diverso produrrebbe un binario diverso e tempi GPU non confrontabili. Ogni job compila i sorgenti sul nodo di calcolo prima di misurare: il build usa `-march=native`, e una compilazione eseguita altrove produrrebbe codice ottimizzato per una CPU diversa da quella su cui si misura. I rank MPI sono lanciati con `mpirun` anziché con `srun`, a causa di un'incompatibilità fra il client PMIx di OpenMPI 4.1.6 e il server PMIx esposto da SLURM (sezione 5.4.3).

2 Frattali

Un frattale è un oggetto geometrico caratterizzato da due proprietà: l'*autosimilarità*, per cui la stessa struttura si ripete, identica o statisticamente analoga, a scale di ingrandimento diverse, e una complessità di dettaglio virtualmente infinita (figura 1a). Un terzo tratto riguarda il modo in cui i frattali vengono generati: non da una descrizione statica, ma dalla *ripetizione iterata di una regola semplice* (figura 1b). Si distinguono tre grandi famiglie. I frattali geometrici si costruiscono applicando ricorsivamente una trasformazione a una figura di partenza, come il triangolo di Sierpiński, l'insieme di Cantor e la curva di Koch (figura 1). I frattali algebrici o *escape-time* si ottengono iterando una funzione sul piano complesso, come l'insieme di Mandelbrot e gli insiemi di Julia. I frattali stocastici, infine, introducono una componente casuale e descrivono molte forme naturali, come alberi, fiumi, coste e nuvole [1].

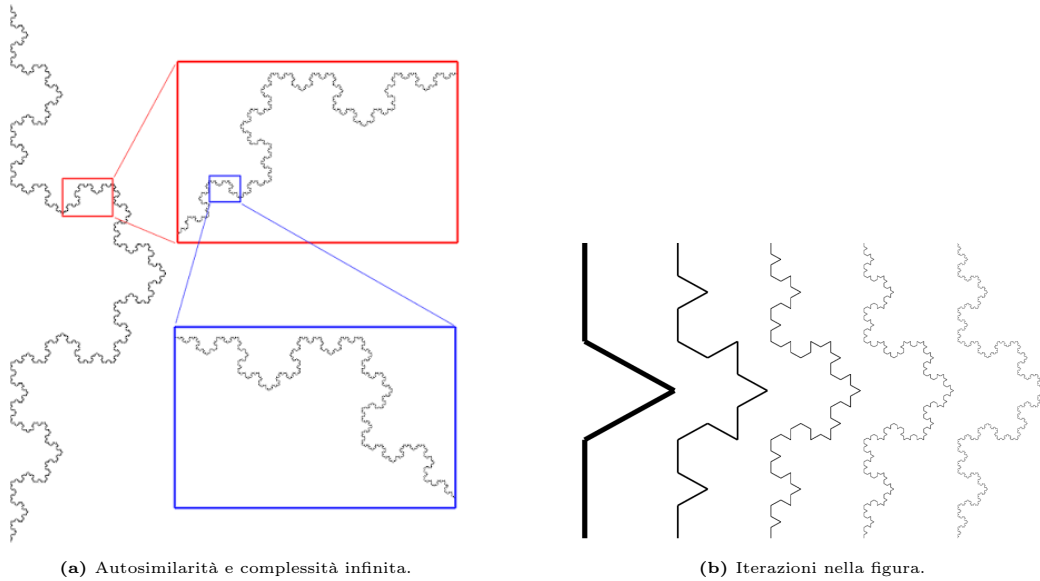


Figura 1: Curva di Koch: il fiocco di neve.

2.1 L'insieme di Mandelbrot

L'insieme di Mandelbrot è l'insieme dei numeri complessi c per i quali la successione

$$\begin{cases} z_0 = 0 \\ z_{n+1} = z_n^2 + c \end{cases}$$

rimane limitata [2].

In accordo con la definizione di frattale, a ogni scala vi si ritrovano copie ridotte dell'intero insieme, immerse in un intrico di filamenti (figura 4). L'autosimilarità è però solo approssimata: le copie sono distorte e circondate da una filigrana ogni volta diversa. La complessità, inoltre, non è distribuita in modo uniforme: le ampie regioni interne ed esterne sono regolari, mentre i dettagli si concentrano sul bordo [3].

Il calcolo del singolo pixel. Nell'implementazione il dominio è discretizzato in una griglia di $W \times H$ pixel. Al pixel di riga i e colonna j , con $0 \leq i < H$ e $0 \leq j < W$, corrisponde il punto del piano complesso

$$c_{i,j} = \left(x_{\min} + \left(j + \frac{1}{2}\right) dx\right) + \left(y_{\max} - \left(i + \frac{1}{2}\right) dy\right) i,$$

dove $dx = (x_{\max} - x_{\min})/W$ e $dy = (y_{\max} - y_{\min})/H$ sono i passi di campionamento: le righe scandiscono la parte immaginaria, le colonne la parte reale, e ogni pixel è rappresentato dal proprio centro.

Come anticipato nell'introduzione, il colore di ciascun pixel dipende soltanto dal proprio c , e i pixel sono quindi indipendenti fra loro. Questa indipendenza convive però con una dipendenza stretta all'interno del singolo pixel. Fissato c , il calcolo dell'orbita¹ è intrinsecamente sequenziale, perché z_{n+1} si calcola solo una volta noto z_n . Il parallelismo del problema risiede quindi *fra* i pixel, non *dentro* ciascuno di essi. Si noti infine la duplice natura di c : distingue un pixel dall'altro ed è, allo stesso tempo, la costante della successione $\{z_n\}$ associata a quel pixel.

Raggio di fuga e criterio di arresto. Per la mappa $z \mapsto z^2 + c$ il valore $r = 2$ è un *raggio di fuga*: non appena $|z_n| > 2$, l'orbita diverge. Se infatti $|c| \leq 2$ e $|z_n| > 2$, allora $|z_{n+1}| \geq |z_n|^2 - |c| > |z_n|$, e il modulo cresce a ogni passo di una quantità che non diminuisce; se $|c| > 2$, già $|z_1| = |c|$ supera il raggio, e un argomento analogo mostra che l'orbita diverge [2]. L'insieme di Mandelbrot M è quindi il luogo dei punti la cui orbita non supera mai il raggio di fuga:

$$c \in M \iff |z_n| \leq 2 \quad \forall n \geq 0. \quad (1)$$

Poiché per i punti di M l'iterazione non terminerebbe mai, l'implementazione usa due condizioni di arresto complementari.

1. *Fuga (escape)*: non appena $|z_n| > 2$, il punto è dichiarato esterno a M . Il numero di iterazioni eseguite è il *tempo di fuga* (escape time) $n_{i,j}$. Dipende soltanto da c ed è in generale tanto più piccolo quanto più c è lontano dal bordo dell'insieme.
2. *Limite massimo di iterazioni* $N_{\max}$: se dopo $N_{\max}$ passi l'orbita non ha superato il raggio di fuga, il punto è considerato, in modo approssimato, interno a M , e il suo tempo di fuga vale $N_{\max}$.

Il tempo di fuga varia quindi di ordini di grandezza da un pixel all'altro: pochi passi per i punti lontani dall'insieme, centinaia o migliaia vicino al bordo, $N_{\max}$ per i punti interni. È questa variabilità l'origine della *distribuzione non uniforme del lavoro* analizzata nelle sezioni successive.

¹Per orbita di un punto c si intende la successione $z_0, z_1, z_2, \dots$ associata a quel pixel. I termini successione e orbita sono intercambiabili, con la sfumatura che il primo enfatizza la lista ordinata di valori, mentre il secondo enfatizza il percorso dinamico nel piano.

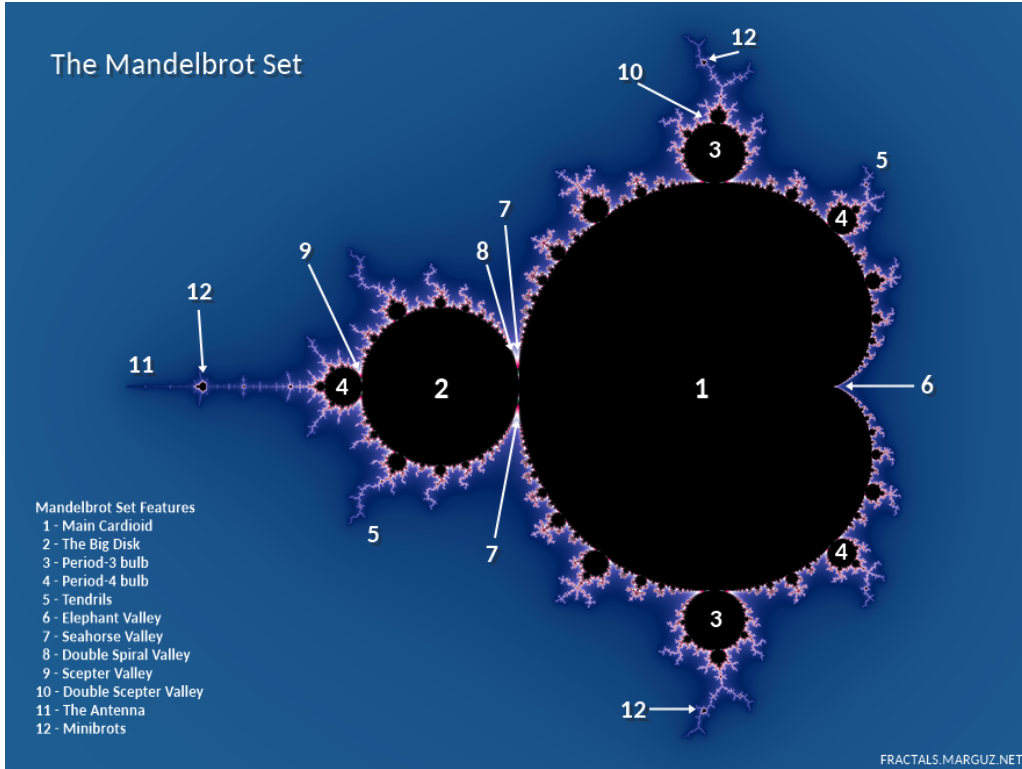


Figura 2: Anatomia dell'insieme di Mandelbrot [4]: (1) cardiode principale; (2) grande disco, il bulbo di periodo 2; (3) bulbo di periodo 3; (4) bulbo di periodo 4; (5) filamenti; (6) valle dell'elefante; (7) valle del cavalluccio marino; (8) valle della doppia spirale; (9) valle dello scettro; (10) valle del doppio scettro; (11) antenna; (12) minibrot.

Struttura dell'insieme. L'insieme di Mandelbrot possiede un'anatomia riconoscibile (figura 2). Il corpo principale è la *cardioide*, la regione in cui l'orbita di 0 è attratta da un punto fisso; a essa è attaccato il *bulbo di periodo 2*, il grande disco a sinistra, in cui l'orbita è attratta invece da un ciclo di periodo 2. Intorno alla cardioide si dispone un'infinità di *bulbi primari*: per ogni punto c di un bulbo di periodo n , l'orbita di 0 è attratta da un ciclo di periodo esattamente n . I bulbi secondari e terziari, attaccati ai primari, hanno periodo multiplo di n .

A ogni bulbo primario è associata un'*antenna*, una struttura filiforme che si prolunga verso l'esterno del bulbo e da cui partono più rami, detti *raggi* (*spokes*). Il numero di raggi dell'antenna principale di un bulbo è uguale al periodo del bulbo [5]: l'antenna del bulbo di periodo 3 ha tre raggi, quella del bulbo di periodo 5 ne ha cinque. I raggi vanno distinti dai *filamenti* (*tendrils*), strutture diramate a ogni scala che si estendono nella regione esterna e portano con sé infiniti mini-Mandelbrot (*minibrot*); i raggi, invece, compongono l'antenna di un singolo bulbo.

Queste strutture determinano dove si concentra il lavoro: le righe che attraversano la cardioide e il grande disco contengono molti punti interni, ciascuno dei quali richiede $N_{\max}$ iterazioni, mentre bulbi, antenne e filamenti portano pixel lenti a fuggire anche nella regione esterna.

Potenze superiori. La ricorrenza $z_{n+1} = z_n^2 + c$ si generalizza elevando z_n a una potenza intera $d \geq 2$:

$$z_{n+1} = z_n^d + c.$$

Gli insiemi che ne risultano conservano la struttura frattale di base, ma cambiano simmetria: l'insieme per $d = 2$ è simmetrico rispetto all'asse reale, quello per $d = 3$ ha simmetria di rotazione di ordine 2, e in generale l'insieme di grado d presenta $d - 1$ bulbi principali equidistribuiti attorno all'origine [6]. Al crescere di d le strutture diventano progressivamente più elaborate, e compaiono nuovi filamenti e bulbi (figura 3).

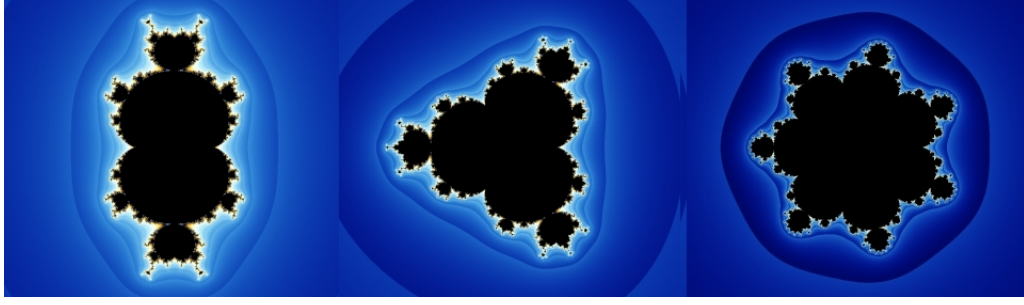
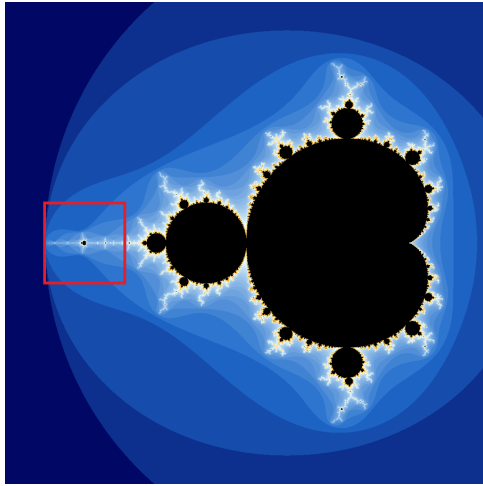
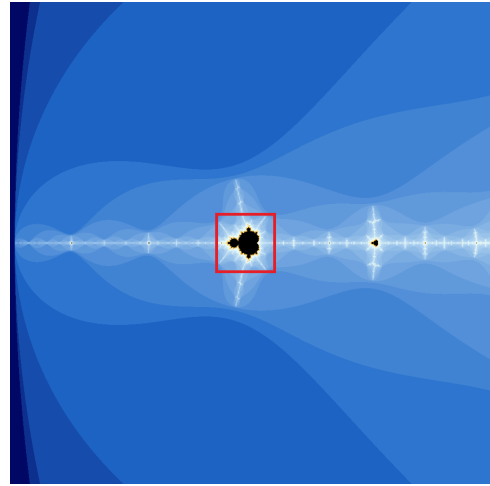


Figura 3: Insiemi di Mandelbrot generalizzati a potenze $d > 2$. Immagine tratta da [6].

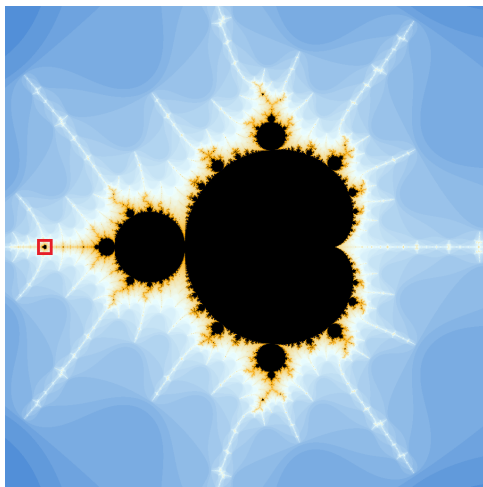
La colorazione escape-time. La colorazione *escape-time* assegna a ciascun pixel esterno un colore in funzione del proprio tempo di fuga e lascia in nero i pixel classificati come interni. Nelle figure di questo lavoro la scala è logaritmica: i punti che fuggono in pochi passi, lontani dall'insieme, sono blu scuro, e le bande si schiariscono verso il bianco e l'ambra avvicinandosi al bordo (figura 4). Le fughe più lente, oltre circa 400 iterazioni, tornano però verso tonalità scure. La forma nera è quindi solo un'approssimazione di M : contiene anche i punti che fuggirebbero dopo più di $N_{\max}$ iterazioni, e la sua accuratezza cresce con $N_{\max}$ e con la risoluzione, al prezzo di un costo di calcolo maggiore. L'alone colorato attorno all'insieme non aggiunge invece informazione sulla forma, perché dipende soltanto dalla palette scelta.



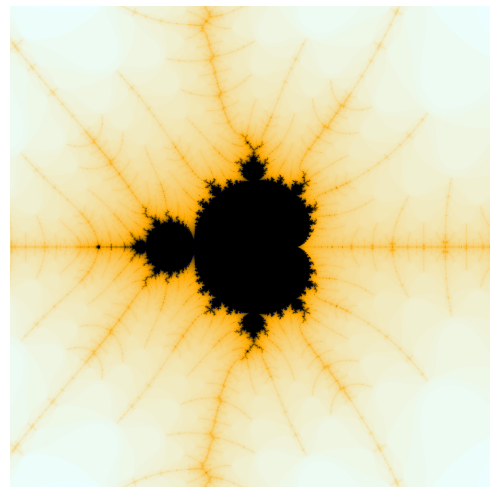
(a) L'insieme completo



(b) Ingrandimento 6×



(c) Ingrandimento 50×



(d) Ingrandimento 1875×

Figura 4: Autosimilarità dell'insieme di Mandelbrot. Quattro finestre annidate l'una dentro l'altra, tutte calcolate dal codice seriale di questo lavoro, a risoluzione 1024×1024 e con $N_{\max} = 1000$ *fisso*: fra un pannello e il successivo cambia unicamente la porzione di piano complesso campionata, indicata dal riquadro rosso. (a) L'insieme completo, $[-2,25; 0,75] \times [-1,5; 1,5]$. (b) La coda dell'antenna, centro $c = -1,7548776$, lato 0,5: sull'asse reale si allineano alcuni nuclei scuri. (c) Uno di essi, centro $c = -1,7608776$, lato 0,06: ricompaiono la cardioide, il bulbo di periodo 2 e la corona di bulbi minori dell'insieme completo. (d) Il nucleo evidenziato sull'antenna della copia precedente, centro $c = -1,7859250$, lato $1,6 \cdot 10^{-3}$: la stessa struttura si ripresenta una terza volta, e si può continuare indefinitamente. Il riquadro in (c) è molto piccolo, perché la copia di (d) è circa 40 volte più piccola di quella che la contiene: è questa gerarchia di scale a produrre la complessità di dettaglio virtualmente infinita. I quattro pannelli condividono la stessa scala di colore, per cui una data tinta corrisponde sempre allo stesso numero di iterazioni: lo sfondo che si scalda progressivamente misura la crescita del tempo di fuga minimo con la profondità dell'ingrandimento, che passa da 1 iterazione in (a) a 25 in (d).

2.2 Pseudocodice

```
1: for each pixel  $(P_x, P_y)$  on the screen do
2:    $x_0 \leftarrow$  scaled  $x$  coordinate of pixel
3:    $y_0 \leftarrow$  scaled  $y$  coordinate of pixel
4:    $x \leftarrow 0, y \leftarrow 0, x_{sq} \leftarrow 0, y_{sq} \leftarrow 0, iteration \leftarrow 0$ 
5:   while  $x_{sq} + y_{sq} \leq 4$  and  $iteration < max\_iteration$  do
6:      $y \leftarrow 2xy + y_0$ 
7:      $x \leftarrow x_{sq} - y_{sq} + x_0$ 
8:      $x_{sq} \leftarrow x^2, y_{sq} \leftarrow y^2$ 
9:      $iteration \leftarrow iteration + 1$ 
10:  end while
11:   $plot(P_x, P_y, palette[iteration])$ 
12: end for
```

dove $z = x + iy$, $c = x_0 + iy_0$ e $max_iteration = N_{max}$. L'algoritmo 2.2 traduce la definizione in istruzioni eseguibili e adotta tre accorgimenti.

- **Test di fuga senza radice quadrata.** Il criterio (1) richiederebbe il modulo $|z_n| = \sqrt{x^2 + y^2}$, e la radice quadrata in virgola mobile è sensibilmente più costosa di moltiplicazioni e addizioni. Poiché entrambi i membri sono non negativi, la condizione si eleva al quadrato senza cambiarne il significato, e la fuga diventa

$$x^2 + y^2 > 4.$$

- **Scomposizione in parte reale e immaginaria.** L'hardware non dispone di un tipo complesso nativo e opera su registri in virgola mobile. Sviluppando

$$z^2 = (x + iy)^2 = x^2 - y^2 + 2ixy,$$

la ricorrenza diventa un sistema di due equazioni reali accoppiate:

$$\begin{cases} x_{n+1} = x_n^2 - y_n^2 + x_0 \\ y_{n+1} = 2x_n y_n + y_0 \end{cases}$$

- **Riuso dei quadrati.** I valori x^2 e y^2 servono sia all'aggiornamento di x sia al test di fuga: memorizzandoli si calcolano una sola volta, e le moltiplicazioni per iterazione scendono da 6 a 4. Il costo per iterazione che ne risulta, $\varphi = 8$ FLOP, è ricavato nella sezione 3.1.2.

Esistono inoltre ottimizzazioni strutturali che sfruttano proprietà geometriche e di simmetria dell'insieme — il pre-screening analitico della cardiode e lo sfruttamento della simmetria assiale — la cui trattazione è rimandata alla sezione 3.4, dopo aver introdotto il contesto implementativo necessario.

3 Metriche e Modelli di Prestazione

3.1 Metriche della baseline sequenziale

Si descrivono qui le grandezze misurate sulla baseline, il modello analitico che le lega al carico computazionale e gli indici che anticipano il comportamento delle versioni parallele. Le grandezze si dividono in due categorie. Il tempo di esecuzione dipende dall'hardware e va misurato su ogni macchina e per ogni configurazione. Il lavoro $\mathcal{W}$, gli indici λ e CoV e il checksum dipendono invece soltanto dagli input ($W \times H$, N_{max} , viewport): sono *deterministici*, grazie ai vincoli di riproducibilità (sezione 3.5), e si calcolano una sola volta dalla baseline, per qualunque paradigma e numero di unità di calcolo.

3.1.1 Tempo di esecuzione

La misura fondamentale è il tempo di calcolo dell'intera immagine, ottenuto con l'orologio monotono della libreria standard, `steady_clock`, che non può essere spostato all'indietro da correzioni di sistema ed è quindi adatto a misurare intervalli. La finestra di cronometraccio racchiude l'allocazione della matrice e i due cicli annidati che scandiscono la griglia, mentre esclude la scrittura su disco e il calcolo delle metriche. L'allocazione è inclusa deliberatamente e non per semplicità: la definizione di speedup richiede che T_1 e $T_{\parallel}(p)$ cronometrino la *stessa* regione di codice, e nelle versioni parallele l'allocazione della matrice condivisa è parte del costo. Il suo peso è comunque marginale: la matrice di riferimento occupa 1024^2 interi a 32 bit, cioè 4 MiB, il cui azzeramento si stima dell'ordine del decimo di millisecondo, contro i 722 ms del calcolo.

Ogni configurazione è eseguita R volte consecutive; si adotta il *minimo* sulle ripetizioni

$$T_1 = \min_{1 \leq t \leq R} T_t^{(\text{obs})} \quad (2)$$

come stima di riferimento. Il rumore di sistema (migrazione fra core, interruzioni, contesa sulla cache) può soltanto aggiungere tempo, mai sottrarlo: la corsa più rapida è la meno contaminata. Mediana e media sono comunque riportate come indicatori della variabilità sperimentale; una divergenza significativa tra T_1 e T_{mean} segnala che almeno una ripetizione è stata perturbata da fattori esterni.

3.1.2 Modello analitico del carico computazionale

Si indica con τ il tempo impiegato da un core per eseguire un'iterazione della ricorrenza introdotta nella sezione 2.1. Si assume τ costante, perché ogni iterazione esegue lo stesso insieme di operazioni in virgola mobile; l'ipotesi è verificata nella sezione 4.3, dove τ resta costante entro l'1,3% su configurazioni il cui lavoro totale varia di un fattore 64. Il tempo di calcolo del pixel (i, j) è $\tau n_{i,j}$, e varia da pixel a pixel per le ragioni viste nella sezione 2.1.

Nel codice sequenziale la variabilità per pixel non influenza il tempo complessivo: un unico processore visita tutti i pixel in successione e il tempo totale è la somma dei costi individuali, indipendente dall'ordine di visita,

$$T_1 = \tau \mathcal{W}, \quad \mathcal{W} = \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} n_{i,j}. \quad (3)$$

Nel caso parallelo, invece, la variabilità determina lo sbilanciamento del carico fra le unità di calcolo (sezione 3.1.3): il tempo di completamento non è più proporzionale alla somma totale $\mathcal{W}$, ma al *carico dell'unità di calcolo più carica*.

Da $\mathcal{W}$ si ricavano il numero totale di FLOP, $F = \varphi \mathcal{W}$ con $\varphi = 8$, e il throughput computazionale² $\Pi = F / (T_1 \cdot 10^9)$ in GFLOP/s. La definizione si estende al caso parallelo, $\Pi(p) = F / (T_{\parallel}(p) \cdot 10^9)$. Poiché F è la stessa quantità per ogni variante, $\Pi(p)$ non è un rapporto rispetto a T_1 : è l'unica metrica *assoluta* del lavoro, e permette di confrontare direttamente CPU e GPU.

Il valore $\varphi = 8$ è ricavato dal corpo del ciclo (algoritmo 2.2): le due moltiplicazioni di $2xy$ e i quadrati x^2 e y^2 danno 4 moltiplicazioni; l'addizione di y_0 , la sottrazione $x^2 - y^2$, l'addizione di x_0 e la somma $x^2 + y^2$ del test di fuga danno 4 addizioni. Il numero di FLOP è quindi

² $\Pi(p)$ misura la velocità dell'intero sistema a p unità di calcolo, non quella per singolo core: quest'ultima si otterrebbe dividendo per p .

calcolato *analiticamente* dalla matrice dei tempi di fuga, e non misurato strumentando il codice, che altererebbe il tempo da misurare.

L'intensità aritmetica che ne risulta colloca il problema nel regime *compute-bound*. Nella configurazione di riferimento (1024×1024 , $N_{\max} = 1000$) ogni pixel richiede in media 247,6 iterazioni, cioè circa $1,98 \cdot 10^3$ FLOP, a fronte di una sola scrittura di 4 byte in memoria: circa $5 \cdot 10^2$ FLOP per byte, un rapporto che cresce con $N_{\max}$. È quindi improbabile che la banda di memoria limiti il calcolo; la verifica sperimentale è nella sezione 4.3, dove τ non cresce con la dimensione della matrice.

3.1.3 Profilo di carico e indici di sbilanciamento

La riga è l'unità di assegnamento delle due parallelizzazioni su CPU: OpenMP distribuisce le iterazioni del ciclo esterno, e MPI assegna righe intere con regole a blocchi, cicliche o dinamiche. La versione CUDA opera invece a granularità di pixel, e per essa l'irregolarità si studia attraverso la divergenza dei warp (sezione 5.6). Si definisce il *carico di riga*

$$w_r = \sum_{j=0}^{W-1} n_{r,j}, \quad 0 \leq r < H, \quad (4)$$

di media $\bar{w} = \mathcal{W}/H$ e deviazione standard σ_w . Poiché la riga è indivisibile, il profilo $\{w_r\}$, disponibile dalla sola esecuzione seriale, permette di calcolare il limite di speedup di qualunque regola che assegni righe intere, prima di implementare il codice parallelo (sezione 3.3).

Si definiscono due indici adimensionali:

$$\lambda = \frac{\max_r w_r}{\bar{w}}, \quad \text{CoV} = \frac{\sigma_w}{\bar{w}}. \quad (5)$$

Entrambi sono proprietà *intrinseche* del profilo $\{w_r\}$, indipendenti da p e dalla regola di assegnamento.

Interpretazione di λ . λ misura il rapporto fra la riga più pesante e la riga media. Poiché nessuna regola può completare il calcolo prima che la riga più pesante sia ultimata, lo speedup di qualunque decomposizione per righe è limitato da

$$S \leq \frac{\mathcal{W}}{\max_r w_r} = \frac{H}{\lambda}.$$

Con $p \ll H$ ogni unità di calcolo riceve molte righe, e questo limite è lontano. Detto W_k il carico dell'unità k (con $0 \leq k < p$) e $\bar{W} = \mathcal{W}/p$ il carico medio, si definisce lo sbilanciamento *effettivo*

$$\lambda_p = \frac{\max_k W_k}{\bar{W}}, \quad (6)$$

che dipende dalla regola di assegnamento. Per le tre regole considerate $\lambda_p \leq \lambda$, con $\lambda_p = \lambda$ nel caso limite $p = H$. Aggregare righe non basta però a bilanciare il carico: se le righe pesanti sono contigue, come in questo problema, finiscono nello stesso blocco e λ_p resta vicino a λ , ed è ciò che le regole cicliche e dinamiche cercano di evitare. Nel modello *compute-only* lo speedup vale p/λ_p (sezione 3.2).

Interpretazione di CoV. Il CoV quantifica la dispersione *globale* del profilo su scala relativa:

- CoV = 0 significa che tutte le righe hanno carico identico, ovvero che il carico è perfettamente uniforme;

- $\text{CoV} = 1$ significa che lo scarto tipico è grande quanto la media stessa, il che equivale a un'irregolarità molto marcata.

I due indici non sono intercambiabili: solo λ , attraverso λ_p , fornisce un limite allo speedup; il CoV descrive la dispersione del profilo e suggerisce quanto possa rendere uno scheduling dinamico a grana fine.

Corrispondenza con il dataset. Nei file di output $\mathcal{W}$ è la colonna W , T_1 è T_{min} , Π è gflops , λ è lambda_row , CoV è cov e p è p . La colonna lambda_block contiene, per ogni run parallela e qualunque sia la regola usata, il λ_p della decomposizione a blocchi contigui: è quindi il limite dello scheduling statico, non delle regole cicliche o dinamiche (sezione 3.3). Nel testo P maiuscola indica la frazione parallelizzabile della legge di Amdahl, e k l'indice delle unità di calcolo.

3.2 Metriche delle implementazioni parallele

Le metriche seguenti dipendono soltanto dai tempi di esecuzione, e non dal meccanismo con cui il parallelismo è realizzato: definirle una sola volta garantisce che i paradigmi siano confrontati sulle stesse basi. Sulla GPU, dove il numero di unità di calcolo non è una variabile controllabile, si usano soltanto tempo e speedup (sezione 5.6.2).

3.2.1 Tempo parallelo

Il tempo della versione parallela su p unità di calcolo è misurato sulla stessa finestra di cronometraggio di T_1 e stimato con il minimo su R ripetizioni:

$$T_{\parallel}(p) = \min_{1 \leq t \leq R} T_t^{(\text{obs})}. \quad (7)$$

Il calcolo termina quando l'unità più carica conclude la propria parte, quindi $T_{\parallel}(p)$ stima il **makespan** $M(p)$, il massimo dei tempi di completamento delle singole unità. Nel modello della sezione 3.1.3

$$M(p) = \tau \max_{0 \leq k < p} W_k, \quad W_k = \sum_{r \in \mathcal{R}_k} w_r, \quad (8)$$

dove $\mathcal{R}_k$ è l'insieme delle righe assegnate all'unità k . Il modello considera soltanto il calcolo, e per questo è detto *compute-only*. Il tempo misurato vi aggiunge l'overhead di comunicazione e sincronizzazione, $T_{\parallel}(p) = M(p) + \epsilon_{\text{overhead}}$, per cui, a meno del rumore di misura, $T_{\parallel}(p) \geq M(p)$.

3.2.2 Speedup ed efficienza

Detto T_1 il tempo della baseline sequenziale (sezione 3.1.1) e $T_{\parallel}(p)$ il tempo della versione parallela su p unità di calcolo, lo **speedup** e l'**efficienza** sono

$$S(p) = \frac{T_1}{T_{\parallel}(p)}, \quad E(p) = \frac{S(p)}{p} = \frac{T_1}{p T_{\parallel}(p)}. \quad (9)$$

Lo speedup misura il fattore di accelerazione, cioè quante volte il codice parallelo è più veloce di quello sequenziale; l'efficienza è la frazione di tempo in cui le unità di calcolo sono impiegate utilmente. $S = p$ ed $E = 1$ corrispondono allo scaling lineare ideale. Sostituendo il modello *compute-only* (8) e $T_1 = \tau \mathcal{W}$:

$$S(p) \approx \frac{\tau \mathcal{W}}{\tau \max_k W_k} = \frac{\mathcal{W}}{\max_k W_k} = \frac{p}{\lambda_p},$$

dove λ_p è definito in (6). Il fattore τ si cancella: nel modello, speedup ed efficienza dipendono soltanto dalla distribuzione delle iterazioni fra le unità di calcolo, non dalla velocità del processore. Poiché $T_{\parallel}(p) \geq M(p)$, p/λ_p è un limite superiore allo speedup misurato.

3.2.3 Legge di Amdahl

A dimensione del problema fissata, detta P la frazione di tempo parallelizzabile e $1 - P$ quella seriale, lo speedup è limitato superiormente da

$$S(p) \leq \frac{1}{(1-P) + \frac{P}{p}} \xrightarrow{p \rightarrow \infty} \frac{1}{1-P}.$$

Come mostrato nella sezione 5, nel codice in esame $1 - P \approx 10^{-4}$: Amdahl prevede uno speedup quasi ideale e fornisce il *limite di riferimento*, ma non spiega gli scostamenti osservati.

3.2.4 Metrica di Karp–Flatt

Per rendere conto di tali scostamenti si ricorre alla frazione seriale *sperimentale* o *empirica*

$$e(p) = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}, \quad (10)$$

ottenuta invertendo la legge di Amdahl a partire dallo speedup misurato. $e(p)$ è ricavata dai tempi effettivi e assorbe in un unico indicatore *tutti* gli effetti *non* ideali: sbilanciamento del carico, comunicazione e sincronizzazione. È pertanto rilevante la sua variazione al crescere di p :

- se $e(p)$ è **costante**, la perdita di efficienza è imputabile a una frazione seriale vera e propria, limite strutturale insensibile all'aumento di p ;
- se $e(p)$ **cresce** con p , la perdita proviene da overhead che peggiora con il grado di parallelismo: costi di comunicazione o sincronizzazione crescenti, oppure saturazione dello speedup per granularità troppo grossa;
- se $e(p)$ **decresce** con p , lo speedup continua a crescere ma con efficienza strutturalmente ridotta: non c'è un overhead crescente, ma uno sbilanciamento il cui tetto scala con p . La metrica tende a zero, non a un valore fisso.

In questo codice la frazione seriale è trascurabile, e le perdite osservate derivano soprattutto da sbilanciamento e overhead: $e(p)$ va quindi letta per il suo andamento, che indica *quale* sia la causa. Per lo sbilanciamento, la grandezza che ne quantifica l'effetto resta λ_p .

3.3 Previsione delle prestazioni parallele dal profilo di carico seriale

Il modello compute-only permette di calcolare i limiti di speedup prima di implementare le versioni parallele. Poiché τ è costante, il makespan è proporzionale al carico dell'unità più carica, che dipende soltanto dal profilo $\{w_r\}$ e dalla regola con cui le righe vengono assegnate. Fissata la regola, $M(p)$ si calcola analiticamente, e con esso $\lambda_p = p M(p)/(\tau \mathcal{W})$ e il limite $S(p) \leq p/\lambda_p$. Si considerano le tre regole implementate nei capitoli successivi:

- **a blocchi**: all'unità k le righe contigue $[kH/p, (k+1)H/p)$;
- **ciclica**: la riga r all'unità $r \bmod p$;
- **dinamica**: coda di task con le righe come unità indivisibili. Nel caso ideale nessuna unità resta inattiva finché c'è lavoro, e il makespan è limitato soltanto dalla quota equa e dalla riga più pesante:

$$M(p) = \tau \max\left(\frac{\mathcal{W}}{p}, \max_r w_r\right).$$

I valori di λ_p e dei limiti sono riportati nella sezione 5.2 e confrontati con gli speedup misurati nella sezione 5.5: è questo confronto a collegare la baseline alle implementazioni parallele. La versione CUDA, che opera a granularità di pixel, ha invece un predittore ricavato dalla matrice degli escape time (sezione 5.6.4).

3.4 Ottimizzazioni avanzate disponibili e scelta della baseline

L'algoritmo di escape-time ammette due ottimizzazioni strutturali che sfruttano proprietà analitiche dell'insieme.

La prima è un *pre-screening geometrico*: cardioide principale e bulbo di periodo 2 ammettono formule chiuse che permettono di classificare un punto come interno prima di entrare nel ciclo iterativo, eliminando le $8 N_{\max}$ operazioni in virgola mobile che ciascuno di quei punti eseguirebbe.

La seconda è lo *sfruttamento della simmetria assiale*: l'insieme è simmetrico rispetto all'asse reale, per cui ogni punto $c = x_0 + iy_0$ e il suo coniugato $\bar{c} = x_0 - iy_0$ hanno lo stesso tempo di fuga. Se il viewport è simmetrico rispetto a $y = 0$, è sufficiente calcolare la metà superiore della griglia e copiare per riflessione, dimezzando il lavoro effettivo.

Entrambe sono implementate e attivabili a tempo di compilazione (`USE_PRUNING`, `USE_SYMMETRY`), ma restano *disattivate* in tutti i benchmark. Con il pre-screening la matrice registrerebbe $N_{\max}$ per punti mai iterati, e $\mathcal{W}$ non misurerebbe più il lavoro eseguito; con la simmetria metà delle righe sarebbe copiata invece che calcolata. In entrambi i casi il profilo $\{w_r\}$ non rifletterebbe più il costo reale, e λ , CoV e le previsioni che ne derivano perderebbero significato proprio sul fenomeno che si intende studiare.

3.5 Artefatto di riferimento e riproducibilità

L'artefatto di riferimento dell'analisi è la matrice dei tempi di fuga $n_{i,j}$ e non l'immagine renderizzata: l'oggetto di studio è il kernel che determina l'escape, mentre la colorazione è una scelta a valle, ininfluyente sulle prestazioni. Per questo l'immagine non viene rigenerata a ogni esecuzione, ma prodotta solo su richiesta esplicita (`--ppm`, `--pgm`, `--raw`); la finestra di cronometraggio esclude ogni scrittura su disco, in modo che l'overhead di I/O non contamini la misura del calcolo puro.

Questa scelta motiva un requisito di correttezza preciso: le quattro implementazioni sviluppate in questo lavoro — baseline sequenziale, OpenMP, MPI e CUDA — calcolano tutte la stessa matrice $n_{i,j}$ a partire dagli stessi input, e devono perciò produrre risultati bit-a-bit identici. La verifica avviene tramite un checksum FNV-1a a 64 bit calcolato sull'intera matrice: *a parità di input* tutte le implementazioni devono restituire lo stesso valore, e anche un solo conteggio diverso produce, in pratica, un checksum diverso.

La riproducibilità bit-a-bit è garantita imponendo la semantica IEEE 754 stretta in fase di compilazione (`-ffp-contract=off`, e il corrispettivo `--fmad=false` per `nvcc`), con esplicita esclusione di `-ffast-math` e più in generale delle ottimizzazioni che alterano l'ordine delle operazioni in virgola mobile: per i pixel prossimi al bordo, anche la differenza di un solo ULP in un risultato intermedio può modificare di ± 1 il conteggio di iterazioni, e dunque il checksum. I flag di compilazione sono definiti in `mandelbrot/src/Makefile`, il checksum in `mandelbrot/src/image_io.cpp`.

4 Implementazione Seriale

L'implementazione sequenziale è il riferimento dell'intero lavoro: produce la matrice degli escape time con cui ogni variante parallela deve coincidere e fornisce il tempo T_1 rispetto al quale sono

definiti speedup ed efficienza. Questa sezione applica gli strumenti introdotti nella sezione 3.1 e ne ricava i valori numerici.

Si discute anzitutto l'effetto dei due parametri di campionamento, la risoluzione $W \times H$ e la soglia iterativa $N_{\max}$, sul carico computazionale e sull'accuratezza dell'immagine; si riporta poi il kernel di escape time; si presentano infine le misure che fissano $\mathcal{W}$, τ , λ , CoV e il checksum della configurazione di riferimento, verificando sui dati il modello $T_1 = \tau \mathcal{W}$. Il profilo di carico e la matrice degli escape time così misurati sono l'unico ingresso delle previsioni sulle versioni parallele, presentate insieme ai rispettivi paradigmi. Tutte le misure sono prese sul kernel *bare*, con pre-screening e simmetria disattivati per le ragioni esposte nella sezione 3.4.

4.1 Impatto della risoluzione e della soglia iterativa

Il carico computazionale del codice seriale dipende da due parametri: la risoluzione $W \times H$ fissa il numero di pixel da calcolare, la soglia $N_{\max}$ limita il numero di iterazioni per pixel. Il lavoro totale $\mathcal{W}$ è il prodotto dei due effetti, cioè il numero di pixel per il numero medio di iterazioni per pixel.

- **Risoluzione $W \times H$.** La griglia determina il numero di task indipendenti: raddoppiando sia W che H , i pixel, e con essi il lavoro, quadruplicano. È l'asse che definisce la dimensione del problema. Una risoluzione maggiore campiona inoltre il frattale più finemente, esponendo la struttura del bordo.
- **Soglia iterativa $N_{\max}$.** Il parametro non altera il numero di pixel, ma limita quanto a fondo si segue ciascuna orbita. I punti esterni a fuga rapida escono ben prima della soglia e ne sono insensibili; i punti interni la raggiungono sempre, e il loro costo cresce linearmente con $N_{\max}$. Una soglia troppo bassa classifica come interni i punti esterni a fuga lenta, che nell'immagine risultano neri pur non appartenendo all'insieme; aumentarla corregge l'errore a un costo concentrato sui punti interni e di bordo.

I dati della sezione 4.3 mostrano che la correzione riguarda una frangia larga circa un pixel lungo il contorno, e che lo sbilanciamento del carico dipende dalla geometria dell'insieme molto più che dalla soglia.

4.2 Frammenti di codice rilevanti

Il codice sorgente completo di tutte le implementazioni, insieme ai job script e agli script di analisi, è disponibile nel repository del progetto³; il codice e i dati citati in questo elaborato corrispondono alla versione b386d12. Il repository separa il kernel di calcolo, unico file riscritto per ciascun paradigma, dal codice condiviso di driver, I/O e metriche, che resta identico in tutte le implementazioni; nel testo si riportano soltanto i frammenti necessari alla discussione, con un rinvio al file completo.

Di seguito si riporta il kernel seriale, tratto da `mandelbrot/src/mandelbrot.cpp`.

Listing 1: Kernel seriale di escape time, senza le righe relative alla simmetria.

```
MandelbrotImage compute_mandelbrot(const Viewport& view, int width, int height,
                                   int max_iter) {
    MandelbrotImage img;
    img.width = width;
    img.height = height;
    img.max_iter = max_iter;
```

³github.com/gaiaLaRocca/parallel_computing_project

```

img.data.resize(static_cast<std::size_t>(width) * height);

const double dx = (view.x_max - view.x_min) / width;
const double dy = (view.y_max - view.y_min) / height;

for (int row = 0; row < height; ++row) {
    const double c_imag = view.y_max - (row + 0.5) * dy;
    for (int col = 0; col < width; ++col) {
        const double c_real = view.x_min + (col + 0.5) * dx;
        img.data[static_cast<std::size_t>(row) * width + col] =
            escape_time(c_real, c_imag, max_iter);
    }
}
return img;
}

int escape_time(double c_real, double c_imag, int max_iter) {
    double z_real = 0.0;
    double z_imag = 0.0;
    double z_real_sq = 0.0;
    double z_imag_sq = 0.0;

    int iter = 0;
    while (iter < max_iter && z_real_sq + z_imag_sq <= ESCAPE_RADIUS_SQ) {
        z_imag = 2.0 * z_real * z_imag + c_imag;
        z_real = z_real_sq - z_imag_sq + c_real;
        z_real_sq = z_real * z_real;
        z_imag_sq = z_imag * z_imag;
        ++iter;
    }
    return iter;
}

```

Il frammento traduce direttamente lo pseudocodice della sezione 2.2, e tre suoi dettagli condizionano il resto del lavoro. Il ciclo esterno scorre le righe e quello interno le colonne, e la matrice è scritta in ordine row-major: è la riga l'unità che OpenMP e MPI distribuiscono fra le unità di calcolo. Ogni pixel è campionato nel proprio centro, il termine $+0,5$, con la stessa espressione che tutte le implementazioni parallele riproducono.

Nella ricorrenza, infine, la parte immaginaria è aggiornata prima di quella reale, e i quadrati sono ricalcolati solo dopo entrambe: ogni aggiornamento usa così i valori dell'iterazione precedente. L'ordine è necessario alla correttezza, e tenerlo identico in ogni implementazione è parte del contratto di riproducibilità (sezione 3.5).

4.3 Risultati e discussione

I risultati di questa sezione costituiscono la baseline per l'analisi dei codici paralleli. Le misure provengono dal job batch 31391, eseguito sul nodo `gnode01`, a cui si aggiungono le due risoluzioni maggiori, 2048^2 e 4096^2 , misurate dal seriale del job 34424 sullo stesso nodo. Ogni configurazione è ripetuta $R = 3$ volte e se ne riporta il minimo secondo (2).

$N_{\max}$	T_1 [s]	W	τ [ns]	pixel interni (frazione) ⁴	λ	CoV	checksum ⁵
500	0,36703	132 831 952	2,763	254 074 (24,230%)	2,7236	0,9485	25a3589a73736fcb
1000	0,72233	259 655 490	2,782	253 372 (24,163%)	2,7638	0,9680	7fba3b93a2fa2397
5000	3,55817	1 271 586 860	2,798	252 858 (24,114%)	2,8031	0,9854	f25625fed7ed57c7

Tabella 1: Baseline seriale al variare di $N_{\max}$, a risoluzione 1024×1024 (job 31391).

Risoluzione	T_1 [s]	$\mathcal{W}$	τ [ns]	λ	CoV	checksum ⁵
512×512	0,18084	64 917 922	2,786	2,7646	0,9679	fb14e831a363787f
1024×1024	0,72233	259 655 490	2,782	2,7638	0,9680	7fba3b93a2fa2397
2048×2048	2,88127	1 038 538 536	2,774	2,7716	0,9681	c44a8b5d6e1fe18b
4096×4096	11,52285	4 154 110 032	2,774	2,7757	0,9681	ed083ccd4b5c0e33

Tabella 2: Baseline seriale al variare della risoluzione, con $N_{\max} = 1000$; le prime due righe provengono dal job 31391, le ultime due dal job 34424.

Il modello $T_1 = \tau \mathcal{W}$ è verificato. Sulle sei configurazioni distinte delle due tabelle il costo per iterazione $\tau = T_1/\mathcal{W}$ resta compreso fra 2,763 e 2,798 ns, con una dispersione dell'1,3%, mentre il lavoro totale varia di un fattore 64; equivalentemente, il throughput resta fra 2,86 e 2,90 GFLOP/s. Il costo per iterazione è dunque una costante della macchina, indipendente dalla dimensione del problema: è questa costanza che autorizza, nelle sezioni parallele, a ragionare sul lavoro $\mathcal{W}$ in luogo del tempo. Ne segue anche la conferma sperimentale del regime compute-bound: se la banda di memoria fosse un vincolo, τ crescerebbe con la dimensione della matrice, mentre da 512^2 a 4096^2 , cioè da 1 a 64 MiB di matrice, varia di meno dello 0,5% e anzi diminuisce leggermente.

La risoluzione governa la quantità di lavoro, non la forma del carico. A ogni raddoppio del lato della griglia il lavoro si moltiplica per 4 entro lo 0,01%, e il tempo per un fattore compreso fra 3,99 e 4,00: la crescita proporzionale al numero di pixel è confermata. Gli indici di sbilanciamento restano invece pressoché immutati su un intervallo di 64 volte i pixel: λ varia fra 2,764 e 2,776 e il CoV fra 0,9679 e 0,9681. Raffinare la griglia campiona più fittamente la stessa funzione di costo senza alterarne la forma relativa, e le previsioni di sbilanciamento ricavate a una risoluzione restano valide, entro lo 0,5%, sull'intero intervallo misurato.

$N_{\max}$ governa il costo per pixel, ma il suo effetto sullo sbilanciamento è debole e saturante. Decuplicando $N_{\max}$ da 500 a 5000, il lavoro cresce solo di un fattore 9,57 e non di 10: la differenza è esattamente il contributo dei punti esterni a fuga rapida, che escono ben prima della soglia e sono perciò insensibili al parametro, come previsto nella sezione 3.1.2. Sugli indici di irregolarità, però, l'effetto è molto più tenue di quanto si potrebbe attendere: a fronte di un fattore 10 su $N_{\max}$, λ passa da 2,7236 a 2,8031 (+2,9%) e il CoV da 0,9485 a 0,9854 (+3,9%), con incrementi decrescenti. Questo risultato corregge un'aspettativa intuitiva: $N_{\max}$ non crea lo sbilanciamento, lo *amplifica marginalmente*. Lo sbilanciamento è già presente in forma quasi completa a $N_{\max} = 500$, perché deriva dalla geometria dell'insieme — le righe che attraversano la cardioide contengono molti punti interni, quelle ai margini quasi nessuno — e la geometria non dipende dalla soglia. Ne discende che $\lambda \approx 2,76$ e $\text{CoV} \approx 0,97$ possono essere trattati come

⁵Il conteggio dei pixel interni — come le immagini discusse nella sezione 4.3 e i dati della Tabella 3 — proviene da un'esecuzione locale della stessa configurazione, non dal job del cluster. Le due esecuzioni sono però sovrapponibili: producono la medesima matrice degli escape time, come attesta l'identità dei checksum e di $\mathcal{W}$ riportati in questa stessa riga. A cambiare fra le due è soltanto il tempo di calcolo, l'unica grandezza dipendente dalla macchina.

⁵I checksum sono riportati in esadecimale, come li stampa l'eseguibile; il dataset li registra in forma decimale. Si osservi che sono quasi tutti distinti, coerentemente con quanto anticipato: il checksum identifica una configurazione, e l'invarianza che verifica è quella *fra le quattro implementazioni a parità di input*, non fra input diversi.

costanti del problema nell'intero intervallo di interesse, e che la scelta $N_{\max} = 1000$ per lo studio parallelo non è restrittiva.

$N_{\max}$ **compra pochissima immagine e molto lavoro.** La colonna dei pixel interni della tabella 1 quantifica l'errore di classificazione discusso nella sezione 4.1. Passando da $N_{\max} = 500$ a $N_{\max} = 5000$ i pixel dichiarati interni scendono da 254 074 a 252 858: appena 1 216 pixel su 1 048 576 cambiano classificazione, lo 0,116% della griglia, mentre il lavoro cresce di un fattore 9,57 e il tempo passa da 0,37 a 3,56 s. Fra 500 e 1000 i pixel corretti sono 702, fra 1000 e 5000 altri 514: il conteggio diminuisce in modo monotono, come previsto, ma con incrementi via via minori. Nell'intervallo esplorato, dunque, alzare la soglia non fa emergere nuove strutture: corregge una frangia larga circa un pixel lungo il contorno.

La verifica visiva conferma il conteggio. Le tre immagini corrispondenti alle righe della tabella, generate con lo stesso codice seriale sulla finestra di riferimento $[-2; 0,5] \times [-1,25; 1,25]$ a 1024^2 , restano indistinguibili a occhio: i 1 216 pixel che cambiano formano una frangia sottile che segue il contorno senza alterarne la forma, e soltanto una quarta immagine, che li evidenzia in rosso, permette di individuarli. Poiché la differenza non è percepibile in stampa, le immagini non sono riprodotte qui; sono disponibili nel repository del progetto, nella cartella `latex_report/Immagini/frattali/`, insieme allo script che le rigenera.

griglia	pixel in frangia	frazione della griglia
512×512	300	0,114%
1024×1024	1 216	0,116%
2048×2048	4 700	0,112%

Tabella 3: Pixel la cui classificazione cambia fra $N_{\max} = 500$ e $N_{\max} = 5000$ (escape time nell'intervallo $[500, 5000]$), sulla finestra di riferimento della Tabella 1 e al variare della risoluzione.

La tabella 3 conferma, sulla frangia, l'invarianza di scala osservata sopra. Al raddoppiare del lato della griglia i pixel di frangia quadruplicano come il totale (fattori 4,05 e 3,86), e la loro frazione resta attorno allo 0,114%, con oscillazioni entro il 2% relativo e senza tendenza monotona. Il dato è coerente con un risultato teorico noto: il bordo dell'insieme di Mandelbrot ha dimensione di Hausdorff, e dunque di box, pari a 2 [7], la stessa dimensione del piano che lo contiene, per cui i pixel di bordo scalano come i pixel totali. Ne discendono due conseguenze. La prima è che raffinare la griglia non migliora la qualità *relativa* della classificazione: aumenta nella stessa proporzione il numero di pixel risolti e quello dei pixel incerti. La seconda offre una lettura geometrica dell'invarianza di λ e CoV osservata al variare della risoluzione: se ogni popolazione definita sul bordo mantiene frazione costante, anche il profilo di carico per riga, che dal bordo dipende, conserva la propria forma relativa.

L'entità dell'irregolarità. La figura 5 mostra il profilo di carico per riga della configurazione di riferimento. Le righe esterne all'insieme costano poche migliaia di iterazioni, quelle che attraversano la cardioide centinaia di migliaia, e le righe costose sono contigue, concentrate al centro della griglia. Con $\text{CoV} \approx 0,97$ la deviazione standard del carico per riga è quasi pari alla sua media, e $\lambda \approx 2,76$ indica che la riga più costosa, la 511, richiede quasi il triplo delle iterazioni della riga media. Un profilo di questo tipo è sfavorevole a qualunque decomposizione in blocchi contigui, che assegnerebbe le righe centrali a pochi processori, e favorevole alle strategie che redistribuiscono il lavoro: è la ragione per cui il confronto fra regole di assegnamento costituisce il nucleo delle sezioni successive.

Dalla baseline alle previsioni. Il profilo $\{w_r\}$ e la matrice degli escape time misurati in questa sezione non si limitano a caratterizzare la baseline: bastano a prevedere il comportamento

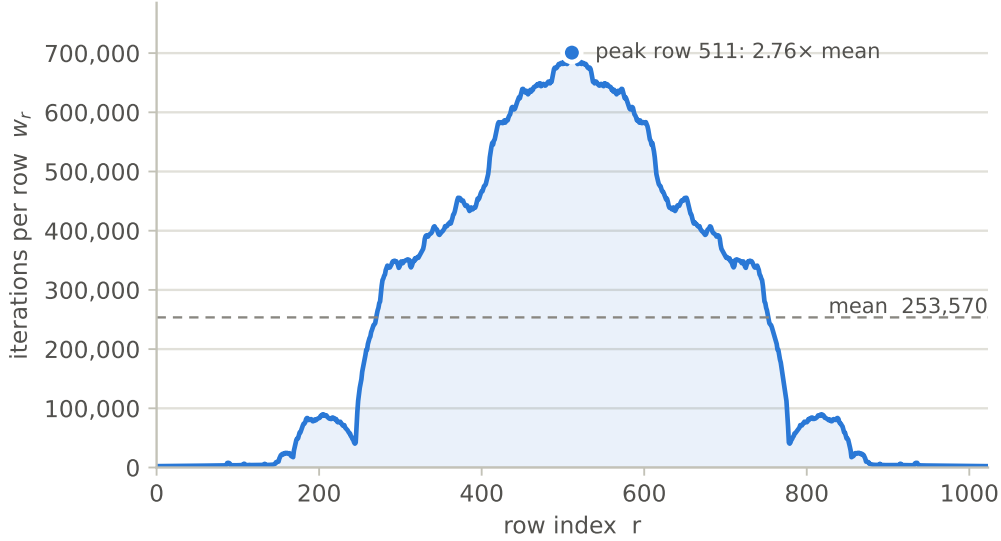


Figura 5: Profilo di carico per riga w_r della baseline seriale (1024^2 , $N_{\max} = 1000$, kernel *bare*). La linea tratteggiata indica la media; il punto segna la riga di picco, che richiede 2,76 volte le iterazioni medie.

delle versioni parallele prima che queste esistano. Le previsioni sono tuttavia presentate insieme ai paradigmi cui si riferiscono, dove i meccanismi che le giustificano sono già stati introdotti: i limiti di speedup delle tre regole di assegnamento delle righe nella sezione 5.2, il predittore della versione GPU nella sezione 5.6.4.

5 Parallelizzazione

5.1 Frazione parallelizzabile e scelte di decomposizione

La frazione parallelizzabile P non misura una proporzione di righe di codice, bensì la quota del tempo seriale T_1 spesa nella parte parallelizzabile. Individuarla richiede di distinguere, nel programma seriale, ciò che costituisce lavoro indipendente da ciò che resta intrinsecamente seriale.

L'unità naturale di parallelismo è il pixel: il kernel della sezione 4.2 lo elabora in tre passi, tutti indipendenti dagli altri pixel.

1. **Calcolo del punto c .** La conversione dell'indice (row, col) nel punto del piano complesso usa soltanto variabili locali, senza dipendenze da altri pixel.
2. **Calcolo dell'escape time.** La ricorrenza è sequenziale *dentro* il pixel, perché l'iterazione $n + 1$ dipende dalla n , ma ogni pixel la esegue in autonomia: il parallelismo risiede *fra* i pixel. È la parte in cui si concentra la quasi totalità del tempo, e quella in cui la variabilità del numero di iterazioni si traduce in sbilanciamento del carico.
3. **Scrittura del risultato.** L'assegnamento `data[row * width + col]` usa un indice univoco per ogni pixel: non vi sono conflitti di scrittura, e la memoria condivisa non richiede né mutua esclusione né sincronizzazione.

Resta fuori dal task la sola frazione seriale: l'allocazione della matrice, l'inizializzazione e la gestione delle strutture dati. Il suo costo è stimato in circa un decimo di millisecondo su 722 ms (sezione 3.1.1), cioè $1 - P \approx 10^{-4}$: a $p = 32$ la legge di Amdahl limita lo speedup a circa 31,9, al di sopra di ogni valore misurato. Le perdite di efficienza osservate nelle sezioni seguenti vanno dunque cercate altrove, nella distribuzione del lavoro.

Quanto alla decomposizione, i due paradigmi CPU non distribuiscono singoli pixel ma righe intere, per le ragioni discusse nella sezione 5.3; la GPU, che dispone di migliaia di thread, assegna invece a ciascuno un solo pixel (sezione 5.6).

5.2 Previsioni dal profilo di carico seriale

Applicando al profilo $\{w_r\}$ misurato nella sezione 4.3 le tre regole di assegnamento definite nella sezione 3.3, si ottengono lo sbilanciamento effettivo $\lambda_p = p M(p)/W$ e il limite $S(p) \leq p/\lambda_p$ di ciascuna decomposizione, prima ancora che il codice parallelo esista.

Le previsioni non sono formulate per implementazione, ma per regola di assegnamento: ciascuna regola è realizzata, con meccanismi diversi, da più di un paradigma (tabella 4), e la stessa riga di previsione va perciò confrontata con le misure di entrambi. La versione CUDA resta fuori da questo quadro: operando a granularità di pixel, non è descritta dal profilo di riga, e il suo predittore, ricavato dalla stessa matrice degli escape time, è presentato nella sezione 5.6.4.

Regola	OpenMP	MPI
blocchi	<code>schedule(static)</code>	decomposizione block
ciclica	— (non misurata)	decomposizione cyclic
dinamica	<code>schedule(dynamic,1)</code>	master-worker

Tabella 4: Corrispondenza fra le tre regole di assegnamento predette dal profilo di carico seriale e i meccanismi che le realizzano nei due paradigmi CPU. La regola ciclica è esercitata dal solo MPI: lo sweep OpenMP non include `schedule(static,1)`, che ne sarebbe l'equivalente a memoria condivisa.

	blocchi		ciclica		dinamica	
p	λ_p	$S \leq$	λ_p	$S \leq$	λ_p	$S \leq$
2	1,0000	2,00	1,0000	2,00	1,0000	2,00
4	1,8798	2,13	1,0001	4,00	1,0000	4,00
8	2,3489	3,41	1,0011	7,99	1,0000	8,00
16	2,5937	6,17	1,0017	15,97	1,0000	16,00
32	2,6619	12,02	1,0095	31,70	1,0000	32,00

Tabella 5: Sbilanciamento effettivo λ_p e limite superiore allo speedup $S \leq p/\lambda_p$, calcolati dal solo profilo di carico seriale (1024×1024 , $N_{\max} = 1000$) per le tre regole di assegnamento delle righe.

La tabella 5 quantifica il costo della scelta di decomposizione a parità di hardware e di numero di processori. Con 32 unità di calcolo, l'assegnamento a blocchi non può superare uno speedup di 12,02: il 62% del tempo di calcolo complessivo va perso in attesa del processore più carico, poiché le righe centrali costose sono contigue e finiscono tutte nello stesso blocco. La sola sostituzione dell'assegnamento contiguo con quello ciclico porta il limite a 31,70, cioè a un'efficienza teorica del 99%: l'interlacciamento distribuisce le righe pesanti fra tutti i processori. Si noti anche il caso $p = 2$, in cui tutte e tre le regole danno $\lambda_p = 1$ esattamente. Per le due regole statiche il motivo è la simmetria del dominio rispetto all'asse reale: la riga r e la riga $H-1-r$ campionano punti coniugati e hanno perciò carico identico, cosicché sia la partizione in due metà contigue sia quella fra righe pari e dispari risultano bilanciate in modo esatto. Per la regola dinamica il motivo è invece che la riga più pesante, pari a 2,76 volte la media, resta molto al di sotto della quota equa $W/2$. Con due soli processori, dunque, la scelta della decomposizione è irrilevante: le differenze fra le strategie emergono solo quando i blocchi diventano abbastanza piccoli da non contenere più un campione rappresentativo del profilo.

Si osserva infine la monotonia annunciata in (6): per l'assegnamento a blocchi λ_p cresce con p ($1,00 \rightarrow 2,66$) avvicinandosi al valore intrinseco $\lambda = 2,76$ misurato nella sezione 4.3, che verrebbe raggiunto nel caso limite $p = H$. Il margine fra 2,66 e 2,76 misura quanta compensazione fra picchi e valli sopravvive ancora aggregando 32 righe per blocco.

5.3 OpenMP: memoria condivisa

Nel modello a memoria condivisa i thread non si scambiano dati in modo esplicito: leggono e scrivono la stessa matrice degli escape time $n(r, c)$, residente in un unico spazio di indirizzamento. Il grado di parallelismo non è quindi fissato dal numero di pixel — che sono milioni — ma dal numero di core del nodo: OpenMP istanzia un pool fisso di thread, e le clausole di schedule non regolano *quanti* thread esistano, bensì la granularità con cui il lavoro viene ripartito tra essi.

Sebbene nella sezione 5 si sia individuato nel singolo pixel l'unità indipendente della computazione, non conviene farne l'unità di *ripartizione*. Assegnare un pixel per volta a ciascun thread introduce due costi che si sommano senza aggiungere parallelismo. Primo, un costo di scheduling: ogni prelievo di una nuova unità richiede un accesso a un contatore condiviso, e per un'unità da poche centinaia di iterazioni tale costo non è trascurabile rispetto al calcolo stesso. Secondo, il *false sharing*: pixel adiacenti sulla stessa riga cadono nella medesima cache line, che rimbalzerebbe tra i core scriventi.

Si ripartisce quindi il ciclo esterno, sulle righe: a ciascun thread è assegnato un gruppo di righe intere. Il costo di scheduling viene così ammortizzato sui pixel della riga, e il false sharing si riduce alle sole righe di confine fra due gruppi, dove l'ultimo pixel di una riga e il primo della successiva possono condividere una cache line: un effetto trascurabile.

La decomposizione per righe rende anche immediata la correttezza. Ogni thread scrive esclusivamente nelle celle delle proprie righe: gli insiemi di celle scritte sono disgiunti, non esiste alcuna *race condition* sui dati e non è necessaria alcuna sincronizzazione in scrittura. Il false sharing residuo è un problema di prestazioni, non di correttezza. La coincidenza del checksum con quello della baseline seriale è la verifica sperimentale di questa assenza di race: una sovrapposizione in scrittura altererebbe la matrice e il checksum non tornerebbe.

5.3.1 Design dello sweep

La domanda progettuale non è quanto parallelizzare, ma *con quale regola assegnare le righe ai thread*. La risposta non è unica: esistono politiche con proprietà di bilanciamento e di coordinamento molto diverse, e confrontarle empiricamente è l'obiettivo dell'esperimento. Lo sweep sistema cinque politiche sullo stesso intervallo di $p \in \{2, 4, 8, 16, 32\}$ thread, a griglia 1024^2 e $N_{\max} = 1000$ fissati: variare le sole leve di scheduling isola l'effetto della politica da quello del carico e del numero di core.

static. Assegna in anticipo a ogni thread un blocco contiguo di H/p righe consecutive. La distribuzione è decisa prima di iniziare: non esiste coda a runtime, il costo di coordinamento è zero. Il prezzo è che un thread che esaurisce il proprio blocco resta inattivo fino a che l'ultimo thread non ha terminato il suo. Poiché le righe che attraversano l'interno dell'insieme costano fino a $N_{\max}$ iterazioni per pixel mentre quelle ai bordi sfuggono in poche iterazioni, i blocchi centrali concentrano il lavoro. **static** è quindi il caso limite in cui il bilanciamento è interamente delegato alla geometria del problema: con un profilo di carico uniforme sarebbe ottimale; con il profilo del Mandelbrot è fortemente sfavorevole. È precisamente questo confronto che motiva la scelta della politica come primo estremo dello sweep.

dynamic con chunk variabile. Invece di assegnare tutto il lavoro in anticipo, il runtime mantiene una coda di chunk: quando un thread ha esaurito le righe che stava elaborando, preleva il chunk successivo. Il ribilanciamento è automatico: i thread veloci — quelli che hanno elaborato righe leggere — riprendono subito lavoro, mentre quelli lenti restano occupati senza lasciare altri in attesa.

La dimensione del chunk governa un compromesso tra due costi opposti. Un chunk piccolo garantisce un bilanciamento quasi perfetto — nessun thread può restare bloccato su un blocco pesante a lungo — ma moltiplica il numero di accessi alla coda condivisa, ciascuno dei quali richiede un’operazione di sincronizzazione interna. Un chunk grande riduce l’overhead di coordinamento, ma riavvicina il comportamento a quello statico: un thread potrebbe caricarsi di k righe tutte pesanti e diventare il collo di bottiglia. Per rendere visibile questo trade-off lo sweep campiona tre valori: $k \in \{1, 16, 64\}$ righe. `dynamic,1` è l’estremo di bilanciamento massimo, `dynamic,64` quello di overhead minimo; `dynamic,16` è il punto intermedio. Il chunk ottimale — quello che minimizza il tempo complessivo — emerge dai dati e costituisce un risultato dell’analisi, non un’ipotesi.

guided. È il compromesso adattivo tra i due estremi. Come `dynamic`, mantiene una coda; a differenza di esso, la dimensione del chunk non è fissa: parte proporzionale al lavoro rimasto e si riduce nel tempo. All’inizio, quando ci sono molte righe disponibili, il chunk è ampio — l’overhead di coda è basso come in `static`. Verso la fine, quando rimangono poche righe e il rischio è che un thread si trovi con un blocco pesante mentre gli altri sono fermi, il chunk si rimpicciolisce, replicando la granularità fine di `dynamic,1` nel momento in cui conta di più. L’idea è ottenere il bilanciamento di `dynamic` pagando una frazione dell’overhead.

5.3.2 Metriche specifiche

OpenMP non introduce metriche proprie: speedup, efficienza e $e(p)$ restano quelle comuni della sezione 3.2, calcolate sui soli tempi. Ciò che è specifico del paradigma è il legame fra la politica di scheduling e lo sbilanciamento previsto dal profilo seriale. In questo quadro CoV è l’indicatore che stabilisce *a priori* se lo scheduling dinamico convenga: un valore elevato segnala una dispersione del carico tale da giustificare il costo di coordinamento, mentre un valore prossimo a zero renderebbe già ottimale la politica statica.

5.3.3 Frammenti di codice rilevanti

Rispetto al kernel seriale il file OpenMP differisce per due sole righe di codice, la direttiva sul ciclo esterno e l’etichetta che identifica il paradigma nel CSV; le funzioni di supporto restano identiche byte per byte, ed è questo a garantire la coincidenza del checksum. Qui si commentano solo le scelte rilevanti; il codice completo è in `mandelbrot/src/mandelbrot_omp.cpp`.

La differenza sostanziale è una singola direttiva sul ciclo esterno:

```
#pragma omp parallel for schedule(runtime)
for (int row = 0; row < computed_rows; ++row) {
    // inner column loop: identical to the serial kernel
    ...
}
```

Non è necessaria alcuna clausola di data-sharing: le variabili dichiarate nel corpo del ciclo sono private per costruzione, mentre la matrice è condivisa ma scritta da ogni thread solo sulle celle delle proprie righe. La scelta implementativa che conta è `schedule(runtime)`: la politica non è fissata nel codice ma letta a esecuzione dalla variabile d’ambiente `OMP_SCHEDULE`, così che un unico binario copra tutte le politiche variando solo il job script, senza ricompilazione. Il confronto tra politiche è dunque uno sweep di configurazioni sullo stesso eseguibile, non binari distinti.

Lo sweep è eseguito da un unico job SLURM su un singolo nodo e si ferma a 32 thread, la capienza di un socket (sezione 1.1). I thread sono ancorati ai core (`OMP_PLACES=cores`, `OMP_PROC_BIND=close`), così che l’intera campagna resti in un solo dominio NUMA e che la

migrazione dei thread non aggiunga rumore al tempo minimo. Il tempo T_1 è misurato nello stesso job con il binario seriale, e non con la versione OpenMP a un thread, e tutte le misure usano il kernel *bare*.

5.3.4 Analisi dei risultati

Il confronto mette a sistema le cinque politiche sullo stesso intervallo di p (da 2 a 32 thread), a risoluzione e $N_{\max}$ fissati. La tabella 6 raccoglie i 25 punti misurati, e la figura 6 ne riporta gli speedup. Il dato d'insieme è la dispersione: a 32 thread, sullo stesso hardware e sullo stesso

Politica	p	$T_{\parallel}(p)$ [s]	$S(p)$	$E(p)$	$e(p)$
seriale	1	0,72563	—	—	—
static	2	0,36076	2,011	1,006	−0,0056
static	4	0,34035	2,132	0,533	0,2921
static	8	0,21280	3,410	0,426	0,1923
static	16	0,11748	6,176	0,386	0,1060
static	32	0,06048	11,999	0,375	0,0538
dynamic,1	2	0,36079	2,011	1,006	−0,0056
dynamic,1	4	0,18043	4,022	1,005	−0,0018
dynamic,1	8	0,09028	8,037	1,005	−0,0007
dynamic,1	16	0,04542	15,977	0,999	0,0001
dynamic,1	32	0,02342	30,988	0,968	0,0011
dynamic,16	2	0,36091	2,011	1,005	−0,0053
dynamic,16	4	0,18041	4,022	1,006	−0,0018
dynamic,16	8	0,09132	7,946	0,993	0,0010
dynamic,16	16	0,04821	15,050	0,941	0,0042
dynamic,16	32	0,03126	23,211	0,725	0,0122
dynamic,64	2	0,36073	2,012	1,006	−0,0057
dynamic,64	4	0,18678	3,885	0,971	0,0099
dynamic,64	8	0,11765	6,168	0,771	0,0424
dynamic,64	16	0,11746	6,178	0,386	0,1060
dynamic,64	32	0,11767	6,167	0,193	0,1351
guided	2	0,36056	2,012	1,006	−0,0062
guided	4	0,26171	2,773	0,693	0,1475
guided	8	0,13250	5,477	0,685	0,0658
guided	16	0,06511	11,144	0,697	0,0290
guided	32	0,03315	21,892	0,684	0,0149

Tabella 6: OpenMP: tempo, speedup, efficienza e frazione seriale sperimentale per ogni combinazione (politica, chunk, thread). Griglia 1024², $N_{\max} = 1000$, kernel *bare*, un socket di un unico nodo, minimo su $R = 5$ ripetizioni. Tutte le righe condividono lo stesso checksum della baseline seriale.

identico carico, la politica migliore è cinque volte più veloce della peggiore. L'ordinamento a $p = 32$ è *dynamic,1* ($S = 30,99$), *dynamic,16* (23,21), *guided* (21,89), *static* (12,00), *dynamic,64* (6,17). Poiché il lavoro da svolgere è identico in tutti i casi — stessa matrice, stesso checksum —, il divario dipende soltanto da come il lavoro è distribuito fra i thread: dall'inattività di chi resta senza righe e, in misura minore, dal costo di coordinamento. A parità di kernel, la scelta della politica vale dunque un fattore cinque.

La granularità può annullare il bilanciamento dinamico. Il caso più istruttivo della tabella è *dynamic,64*, che a 8, 16 e 32 thread restituisce lo stesso tempo a tre cifre significative (0,11765, 0,11746, 0,11767 s): raddoppiare e poi riquadruplicare i thread non produce alcun guadagno. La spiegazione è quantitativa e si legge nella tabella di previsione 5. Con chunk da 64 righe su 1024 esistono soltanto 16 chunk, ciascuno dei quali è un blocco contiguo; il makespan

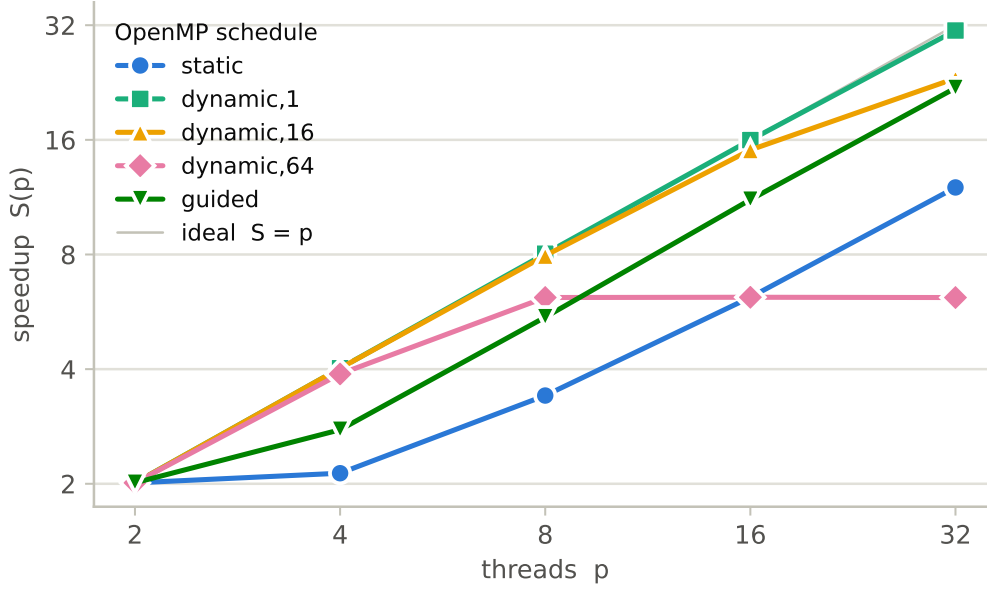


Figura 6: Speedup delle cinque politiche OpenMP in funzione del numero di thread, su assi logaritmici in base 2, con la retta ideale $S = p$. I dati sono quelli della tabella 6.

non può scendere sotto il chunk più pesante, e il peso del più pesante fra 16 blocchi contigui è precisamente ciò che misura λ_p a $p = 16$, da cui il tetto 6,169. Il valore misurato è 6,167. La soglia cade dove la quota equa $\mathcal{W}/p$ scende sotto il chunk massimo, cioè attorno a $p = 7$: da lì in avanti aggiungere thread non cambia nulla, e a 32 thread se ne stanno usando in pratica sei.

Se ne ricava una regola generale utile anche altrove: *per uno scheduling dinamico a chunk fissi di c righe, il tetto allo speedup è quello della decomposizione a blocchi con H/c blocchi, indipendentemente dal numero di thread.* Uno scheduling dinamico con grana troppo grossa degenera in uno scheduling statico con pochi blocchi, e ne eredita per intero lo sbilanciamento pagando in più il costo della coda.

Il chunk unitario e il costo del coordinamento. All'estremo opposto, `dynamic,1` raggiunge 30,99 su 32 thread, cioè il 96,8% dell'ideale, con 88,7 GFLOP/s contro i 2,86 del seriale. Le 1024 righe formano una coda abbastanza fine da assorbire completamente l'irregolarità: la sua $e(p)$ resta sotto 0,0011 ovunque, da uno a due ordini di grandezza più piccola di quella dello statico. Il costo di coordinamento non è però nullo, e diventa visibile proprio dove il bilanciamento è perfetto: fra $p = 16$ e $p = 32$ l'efficienza scende da 0,999 a 0,968, ed è l'unico punto della campagna in cui `dynamic,1` si scosta apprezzabilmente dall'ideale. Con 1024 prelievi da un contatore condiviso fra 32 thread, la contesa sulla coda comincia a farsi sentire. Il confronto con `dynamic,16`, che a 32 thread si ferma a 23,21, mostra che il chunk deve essere abbastanza piccolo da bilanciare: qui il guadagno di bilanciamento fra 16 righe e una riga supera il costo aggiuntivo della coda, perché il carico è molto irregolare ($\text{CoV} \approx 0,97$). Su un carico uniforme ci si attenderebbe l'esito opposto, ed è questo il senso di CoV come indicatore a priori.

guided perde per costruzione, non per overhead. La politica `guided` mostra un'efficienza notevolmente stabile (0,693, 0,685, 0,697, 0,684 da 4 a 32 thread): perde circa il 31% e lo perde allo stesso modo a ogni grado di parallelismo. Un difetto di overhead crescerebbe con p ; una frazione seriale darebbe una $e(p)$ costante, mentre qui $e(p)$ decresce (0,1475 $\rightarrow$ 0,0149). Il difetto è strutturale: `guided` inizia assegnando chunk proporzionali alle righe rimanenti, cioè blocchi contigui grandi, e i blocchi contigui in questo problema ereditano la montagna centrale. I chunk

fini, che sarebbero quelli capaci di bilanciare, arrivano solo alla fine, quando non resta lavoro con cui compensare. Il caso $p = 4$ è il più netto: 2,773 contro i 4,022 di `dynamic,16`, un divario del 31% fra due politiche entrambe dinamiche. `guided` è progettata per ridurre l’overhead di coda su carichi *quasi* bilanciati; qui il carico è tutt’altro, e la sua euristica lavora contro il problema.

Throughput assoluto. Le metriche viste finora sono tutte *relative*: S , E ed e sono rapporti a T_1 , e per costruzione non dicono nulla su quanto valga T_1 stesso. Il throughput $\Pi(p)$ definito nella sezione 3.1.2 è l’unica colonna assoluta della campagna, perché il numeratore $F = \varphi \mathcal{W} = 2,077 \cdot 10^9$ FLOP è la medesima quantità analitica per ogni variante — discende dalla matrice degli escape time, non dall’implementazione. A 32 thread le cinque politiche danno

	<code>dynamic,1</code>	<code>dynamic,16</code>	<code>guided</code>	<code>static</code>	<code>dynamic,64</code>
$\Pi(32)$ [GFLOP/s]	88,71	66,45	62,67	34,35	17,65

contro i 2,86 GFLOP/s del kernel seriale. Come metrica di scaling Π non aggiunge nulla — il rapporto $\Pi(p)/\Pi(1)$ è lo speedup, per definizione — ma rende visibile in valore assoluto ciò che la colonna S esprime in rapporto, e soprattutto è la valuta comune con cui la sezione 5.6 confronterà CPU e GPU, dove il rapporto con il picco teorico dell’hardware diventa la domanda centrale. Qui quel confronto non è proponibile, perché il picco del nodo di calcolo non è stato caratterizzato, e non lo si stima: Π resta un termine di paragone *fra le misure*, non con il silicio.

Il caso che il throughput illustra meglio è `dynamic,64`, la cui curva Π si inchioda a 17,66, 17,68, 17,65 GFLOP/s a 8, 16 e 32 thread: il tetto di granularità discusso sopra, espresso in valore assoluto.

Lo statico satura il limite predetto. I dati di `static` si confrontano con il limite p/λ_p calcolato dal profilo seriale; la verifica quantitativa è riportata nella sezione 5.5, dove lo stesso confronto è esteso alle decomposizioni MPI. Anticipando il risultato: l’accordo è entro lo 0,6% su tutti e cinque i punti, confermando che la perdita di efficienza è interamente attribuibile allo sbilanciamento strutturale e non a overhead di OpenMP.

Lettura di Karp–Flatt. Delle tre firme classificate nella sezione 3.2 le cinque politiche ne esibiscono due soltanto, e l’assente è già un risultato: nessuna mostra $e(p)$ *costante*, cioè in nessun caso la perdita è imputabile a una frazione seriale genuina — coerentemente con un kernel che non ne contiene alcuna. Tutto ciò che $e(p)$ registra qui è sbilanciamento oppure coordinamento, e la direzione della metrica basta a separarli, purché la si legga insieme alla forma del tetto e non da sola.

Politica	$E(p), p \geq 4$	$e(p)$	Meccanismo
<code>static</code>	0,533 $\rightarrow$ 0,375	decresce, 0,2921 $\rightarrow$ 0,0538	sbilanciamento strutturale
<code>guided</code>	$\approx$ 0,690 costante	decresce, 0,1475 $\rightarrow$ 0,0149	sbilanciamento strutturale
<code>dynamic,1</code>	1,005 $\rightarrow$ 0,968	cresce, $-0,0018 \rightarrow$ 0,0011	contesa sulla coda
<code>dynamic,16</code>	1,006 $\rightarrow$ 0,725	cresce, $-0,0018 \rightarrow$ 0,0122	contesa sulla coda
<code>dynamic,64</code>	0,971 $\rightarrow$ 0,193	cresce, 0,0099 $\rightarrow$ 0,1351	tetto di granularità

Tabella 7: OpenMP: classificazione delle cinque politiche secondo l’andamento di $e(p)$. L’efficienza è riportata sull’intervallo $p \geq 4$, che esclude il punto $p = 2$ affetto dallo scarto di T_1 discusso più avanti. Nessuna riga esibisce $e(p)$ costante: la frazione seriale nominale del kernel è nulla.

Le politiche a $e(p)$ decrescente. `static` e `guided` condividono la stessa firma: un’efficienza che si stabilizza su un valore $E_\infty < 1$ e una $e(p)$ che, sostituendo $S = p E_\infty$ in (10), vale

$$e(p) = \frac{1/E_\infty - 1}{p - 1} \xrightarrow{p \rightarrow \infty} 0.$$

guided ne è l'esempio più puro, perché la sua efficienza è costante già a $p = 4$ (0,693, 0,685, 0,697, 0,684): con $E_\infty = 0,690$ la formula predice 0,1500, 0,0643, 0,0300, 0,0145 contro i misurati 0,1475, 0,0658, 0,0290, 0,0149. È la conferma quantitativa di quanto osservato sopra: **guided** perde il 31% e lo perde allo stesso modo a ogni grado di parallelismo, il che è incompatibile con un difetto di overhead.

Per **static** l'efficienza sta ancora scendendo verso il proprio asintoto $1/\lambda = 0,362$, e conviene perciò usare la forma esatta. La politica satura il proprio limite (sezione 5.5), cioè $S(p) = p/\lambda_p$, da cui

$$e(p) = \frac{\lambda_p - 1}{p - 1} \quad (0,2933, 0,1927, 0,1062, 0,0536 \text{ per } p = 4, 8, 16, 32),$$

che riproduce i valori misurati entro lo 0,5%. Poiché λ_p è limitato superiormente dal valore intrinseco $\lambda = 2,76$, anche qui la metrica tende a *zero*. Non c'è dunque alcun tetto fisso che lo statico stia saturando: il valore 12,02 è il limite *alla singola taglia* $p = 32$, non un asintoto, perché il tetto della decomposizione a blocchi vale p/λ_p e cresce con p , fino a $H/\lambda \approx 370$ nel caso limite $p = H$. Che $e(32) = 0,0538$ sia già sceso *sotto* $1/12,02 \approx 0,083$ è la verifica diretta di questo punto, e sarebbe impossibile per una metrica che vi convergesse dall'alto.

Le politiche a $e(p)$ crescente. Per **dynamic,1** e **dynamic,16** la metrica parte da zero e cresce di poco (0,0011 e 0,0122 a $p = 32$): è il costo di coordinamento della coda condivisa, che aumenta con il numero di thread che vi accedono. Non c'è qui alcun tetto — l'efficienza resta alta — e infatti i valori restano di uno o due ordini di grandezza sotto quelli delle politiche sbilanciate.

dynamic,64 appartiene alla stessa direzione ma per una ragione diversa, ed è il solo caso dello sweep in cui il tetto è genuinamente fisso. I 16 chunk da 64 righe non hanno un massimo ma *due*, di peso identico (2,59 volte la media): il profilo di carico è simmetrico rispetto all'asse reale, e i chunk delle righe 448–511 e 512–575 lo attraversano specularmente. Ciascuno dei due pesa il 16,2% di $\mathcal{W}$, e perché il makespan scenda al tempo di uno di essi occorre che i restanti $p - 2$ thread smaltiscano gli altri quattordici chunk — il 67,6% del lavoro — prima che quel tempo sia trascorso, cioè $p \geq 6,17$: è la stessa soglia ricavata sopra. Da lì in avanti lo speedup si inchioda a $S_{\max} = 6,169$ e il makespan diventa praticamente indipendente da p ($p = 8$ è il primo punto misurato oltre la soglia, e vi si trova già a meno dello 0,5%). Con S costante in (10) si ottiene

$$e(p) = \frac{1/S_{\max} - 1/p}{1 - 1/p} \quad (0,0424, 0,1062, 0,1351 \text{ per } p = 8, 16, 32),$$

che converge al reciproco $1/S_{\max} \approx 0,162$ *dal basso*.

È la classificazione della sezione 3.2 osservata sui dati: le due politiche peggiori dello sweep, **static** e **dynamic,64**, stanno agli estremi opposti pur fallendo entrambe, ed è l'andamento di $e(p)$ a distinguerne la causa.

Due avvertenze sulla lettura dei dati. A $p = 2$ tutte e cinque le politiche danno $S \approx 2,011$, cioè leggermente *superlineare*, con $E > 1$ e $e(p)$ negativa. Lo scarto è di 4 ms su 726 ed è identico su tutte le politiche, quindi non è un effetto di scheduling: la causa è la variabilità di T_1 stesso fra job distinti. Lo stesso binario seriale, sulla stessa configurazione e sullo stesso nodo, misura 0,72563 s nel job OpenMP e 0,72233 s nel job della baseline (tabella 1): 3,3 ms di differenza, cioè da sola quasi tutta l'eccedenza. Adottando la seconda misura lo speedup a $p = 2$ scende a 2,002 e la superlinearità sparisce. Il valore non va dunque interpretato come un guadagno superlineare genuino, ma come il rumore residuo fra job distinti su un nodo condiviso.

La tabella conserva comunque il T_1 misurato nello stesso job dei $T_{||}(p)$: stesso job significa stesso nodo nello stesso stato, mentre adottare la misura più veloce fra job diversi nasconderebbe

l'anomalia scegliendo il denominatore più comodo. Per la stessa ragione lo script di merge abbina ogni misura alla baseline del proprio job: con il minimo globale, l'arrivo dello sweep MPI ($T_1 = 0,72021$ s) avrebbe spostato dello 0,75% tutti gli speedup di questa tabella senza che alcuna misura OpenMP fosse ripetuta.

La seconda avvertenza riguarda il rumore di sistema e giustifica a posteriori la scelta del minimo su ripetizioni. Per `guided` a 32 thread il tempo medio sulle cinque ripetizioni è 0,0496 s contro un minimo di 0,0331 s e una mediana di 0,0339 s: la media supera il minimo del 50%, ma è la mediana a rivelare la struttura del disturbo. Quattro ripetizioni su cinque stanno attorno al minimo, e perché la media salga a 0,0496 s la quinta deve essere durata circa 0,11 s, cioè *tre volte e mezzo* il minimo: un singolo episodio isolato e non un rallentamento diffuso, presumibilmente per contesa su banda di memoria o cache di ultimo livello con un altro job presente sul nodo. La media avrebbe attribuito a `guided` un difetto che non ha; il minimo, essendo il rumore additivo, ha isolato il tempo di calcolo genuino.

Sintesi. Il filo *predetto contro misurato* si chiude su tre livelli. La correttezza è verificata dal checksum, identico su tutte e 26 le righe e coincidente con quello della baseline seriale: nessuna politica ha alterato il risultato, il che conferma sperimentalmente l'assenza di corse in scrittura. Lo sbilanciamento predetto in sede seriale è verificato dalla saturazione dello statico sul proprio limite entro lo 0,6%, come dettagliato nella sezione 5.5. E il valore diagnostico del CoV $\approx 0,97$ è confermato dal fatto che lo scheduling dinamico a grana fine, che su un carico uniforme sarebbe stato uno spreco, recupera qui quasi per intero il 62% di efficienza che la decomposizione statica perde.

Resta aperta una domanda che questa sottosezione non può chiudere da sola: quanto di ciò che si è visto dipenda dalla *politica* e quanto dal *modello di memoria*. La risposta richiede di eseguire la stessa identica politica sotto un paradigma diverso, ed è il confronto che la sezione 5.4.4 conduce mettendo `dynamic, 1` a fronte del master-worker MPI a concessione unitaria, sullo stesso nodo (tabella 9). Anticipandone l'esito: i due meccanismi si equivalgono entro circa l'1% a 32 unità, e il divario che resta non è di prestazioni ma di complessità implementativa.

5.4 MPI: memoria distribuita

Il passaggio a MPI cambia il vincolo fondamentale, non la decomposizione. La grana resta la riga, per le stesse ragioni della sezione 5.3, ma i processi non condividono più uno spazio di indirizzamento: ciascun rank possiede una propria copia parziale della matrice degli escape time, e il risultato completo esiste solo dopo un atto esplicito di comunicazione. Ne discendono le due differenze che governano l'intera sottosezione. Primo, la matrice va *assemblata*: nei due schemi statici con una `MPI_Gatherv` finale su rank 0, nello schema dinamico incrementalmente, riga per riga, man mano che i risultati arrivano. Secondo, e più importante, il ribilanciamento del carico non è più gratuito: sotto OpenMP un thread inattivo prelevava lavoro da una coda in memoria condivisa a costo trascurabile, mentre qui ogni atto di ridistribuzione è un messaggio, potenzialmente attraverso la rete. È questo prezzo a rendere il confronto fra schemi la vera domanda del paradigma.

Il parallelismo resta quello per righe, ma l'unità è ora il processo e non il thread; il numero di unità di calcolo è spazzato sugli stessi valori (2, 4, 8, 16, 32) usati per OpenMP, e griglia e $N_{\max}$ restano fissi (1024^2 , 1000), così che il confronto fra paradigmi avvenga sullo stesso carico di lavoro. Non si sfrutta la simmetria, coerentemente con tutti gli altri benchmark di scaling: dimezzerebbe il lavoro senza cambiare la matrice risultante, ma renderebbe metà delle righe una copia anziché un calcolo, falsando il profilo $\{w_r\}$ su cui poggia l'intera analisi di sbilanciamento.

5.4.1 Design dello sweep

La domanda progettuale non è quanto parallelizzare, ma *con quale regola assegnare le righe ai rank*. A differenza di OpenMP, dove la scelta era fra politiche di scheduling dello stesso pool di thread, qui le regole differiscono anche nel volume di comunicazione che generano: è possibile bilanciare meglio pagando più messaggi, oppure bilanciare altrettanto bene senza aggiungerne. Questo asse è ciò che i tre schemi esplorano.

block. Il rank r possiede un intervallo contiguo di H/p righe. La distribuzione è nota a tutti prima di iniziare: non esiste dialogo, e la comunicazione si riduce a un'unica MPI_Gatherv finale. Poiché, per la viewport scelta, il corpo del frattale occupa le righe centrali della griglia e le righe di bordo contengono solo punti esterni che sfuggono in poche iterazioni, i rank centrali concentrano quasi tutto il lavoro. La riga più leggera è la riga 0 (2 281 iterazioni totali, circa 2 per pixel) e la più pesante la riga 511, al centro (700 806, circa 684 per pixel): un rapporto di $307\times$. I rank che ricevono il primo e l'ultimo blocco sono necessariamente veloci per costruzione; i rank centrali portano da soli la maggior parte del lavoro. È il caso di massimo sbilanciamento con comunicazione minima, e il suo limite $S \leq p/\lambda_p$ è calcolabile dal solo profilo seriale prima ancora che il codice MPI esista.

cyclic. La riga r va al rank $r \bmod p$. Ogni rank riceve righe sparse su tutta l'immagine — un po' di bordo, un po' di centro — e il carico si media da sé, perché il pattern di fuga varia su una scala spaziale molto più grande della singola riga. L'assegnazione resta *statica*: ogni rank sa già quali righe possiede, non esiste coda residua, e la comunicazione è la stessa unica gather del block — a meno di un riordino locale su rank 0 per de-interlacciare il buffer, che non è un messaggio ma non è nemmeno gratuito, come mostreranno i risultati.

Ciò che rende il cyclic interessante non è soltanto che batta il block: è che *esce dall'asse* bilanciamento-comunicazione su cui si trova il dinamico. Lungo quell'asse, ridurre il chunk avvicina al bilanciamento ma moltiplica i messaggi; il cyclic ottiene un bilanciamento quasi equivalente al dinamico con zero messaggi aggiuntivi, perché è l'interlacciamento statico a mediare il carico, non la coordinazione a runtime. La domanda aperta che l'esperimento deve rispondere è se questo vantaggio sia sufficiente a battere o pareggiare il master-worker nonostante il riordino.

dynamic (master-worker). Il rank 0 non calcola nulla: tiene le righe non distribuite come coda di lavoro, innesca ciascun worker con una riga e poi alimenta con la successiva il primo worker che riporta un risultato. L'assegnazione si adatta al costo reale di ogni riga — chi è bloccato su una riga pesante semplicemente non viene rifornito — e il bilanciamento è perfetto per costruzione. Il prezzo è una coppia di messaggi richiesta/risposta per ogni riga. Questo schema è il limite opposto al block: bilanciamento massimo, overhead di comunicazione massimo.

È importante notare che, al crescere del chunk, il master-worker non degenera nel cyclic ma nel block: con chunk pari a H/p righe, dove p è il numero di worker, ogni worker riceve un unico blocco contiguo, che è esattamente la decomposizione a blocchi contigui. Il cyclic non è un punto sull'asse chunk-overhead: è un meccanismo ortogonale che non occupa alcuna posizione su quell'asse.

Grana della concessione e conteggio delle unità. La concessione è fissata a una riga, senza sweep sul chunk, per tre ragioni. Un chunk intermedio si colloca fra estremi già misurati: comunica più del cyclic, che non aggiunge messaggi, e bilancia meno del chunk unitario. In secondo luogo l'estremo è il punto più informativo: accoppia il miglior bilanciamento possibile con

il massimo costo di coordinamento, ed è precisamente il confronto che lo schema esiste per esporre. Infine l'asse del chunk è già spazzato dove costa poco, nei run OpenMP `dynamic, {1, 16, 64}`; tenendo il chunk unitario da entrambi i lati, i due scheduler dinamici diventano la stessa politica su due modelli di memoria diversi, e il loro divario isola il costo del passaggio di messaggi rispetto all'incremento di un contatore condiviso.

Poiché il master non calcola, per lo schema dinamico p indica il numero di *worker* e non di processi: gli sweep girano su $np \in \{3, 5, 9, 17, 33\}$ per mantenere le unità di calcolo a $\{2, 4, 8, 16, 32\}$, allineate agli altri schemi e ai thread OpenMP. Va dichiarato che la contabilità è leggermente generosa verso il master-worker: il cluster alloca $p + 1$ core, uno dei quali non calcola; speedup ed efficienza sono per unità di calcolo, e il costo reale in risorse è $p + 1$ processi.

5.4.2 Metriche specifiche

Alle metriche comuni della sezione 3.2 MPI aggiunge una grandezza propria: la *frazione di comunicazione*, cioè la quota di $T_{\parallel}(p)$ spesa a muovere dati anziché a calcolare. È la metrica che dà corpo al costo introdotto dalla memoria distribuita, e la sua definizione operativa differisce fra schemi statici e dinamico.

Negli schemi statici la comunicazione è un unico MPI_Gatherv finale, e il suo tempo è misurato *dopo* una barriera. La scelta è deliberata: senza barriera il rank in ritardo farebbe apparire dentro la gather l'attesa dovuta allo sbilanciamento, gonfiando `comm_fraction` con un tempo che non è trasferimento di dati. Con la barriera l'inattività resta parcheggiata prima dell'operazione — e comunque dentro la regione cronometrata, quindi $T_{\parallel}(p)$ resta onesto — mentre `comm_fraction` misura solo movimento genuino di dati. Lo sbilanciamento è riportato altrove, da λ_p e dallo speedup; e tenere le due cose separate è ciò che rende leggibile il confronto.

Nello schema dinamico la definizione è necessariamente diversa. Il master comunica il 100% del tempo per costruzione — è la sua sola funzione — quindi il suo tempo non direbbe nulla sulla frazione di coordinamento pagata *dal calcolo*. Si misura invece, per ciascun worker, la frazione del proprio tempo trascorsa dentro le chiamate MPI: l'invio del risultato di una riga e l'attesa bloccante del comando successivo. Questa attesa è inclusa deliberatamente, perché contiene sia la latenza del messaggio sia la contesa quando più worker riportano insieme — che è precisamente il prezzo della coordinazione dinamica. Il valore medio dei worker è raccolto con una MPI_Reduce su rank 0 e riportato come `comm_fraction`. È l'unico degli schemi per cui questa colonna diventa una metrica significativa: negli statici la frazione è attesa prossima a zero e serve come controllo, nel dinamico è il numero che quantifica il costo del self-scheduling.

5.4.3 Frammenti di codice rilevanti

Come per OpenMP, un solo binario copre tutti gli schemi: la variabile d'ambiente `MPI_DECOMP` seleziona a esecuzione fra `block`, `cyclic` e `dynamic` (default `block`), e il job passa l'etichetta corrispondente a `--schedule` per il CSV. Il codice completo del kernel si trova nel file `mandelbrot/src/mandelbrot_mpi.cpp`, mentre lo strato driver descritto più avanti si trova nel file `mandelbrot/src/main.cpp`.

La differenza fra i due schemi statici è una sola espressione — quale riga globale corrisponda all' i -esima riga locale:

```
// block:  global row = row_displs[rank] + i    (contiguous range)
// cyclic: global row = rank + i*size            (strided)
const int row = (decomp == Decomp::Block) ? (row_displs[rank] + i)
                                              : (rank + i * size);
```

È il frammento più significativo della sottosezione: il cyclic ottiene gran parte del bilanciamento del master-worker cambiando un indice. I conteggi di righe per rank sono identici nei due casi (base righe ciascuno, le prime `rem` ne prendono una in più); cambia solo *quali* righe un rank possiede. Il block raccoglie direttamente nella matrice finale, essendo i suoi dati già contigui; il cyclic raccoglie in un buffer compattato che il rank 0 poi de-interlaccia, un riordino locale in memoria che non aggiunge alcun messaggio. La simmetria fra i due schemi è però meno perfetta di quanto la singola espressione suggerisca: quel riordino è una copia seriale dell'intera matrice, eseguita da un solo rank e di costo indipendente da p . Non entra in `comm_fraction` — non è comunicazione — ma entra in $T_{\parallel}(p)$, ed è l'unico costo del cyclic assente negli altri due schemi; se ne discute il peso nell'analisi dei risultati. Sarebbe eliminabile ricevendo direttamente in posizione con un tipo derivato `MPI_Type_vector` di passo `size`, al prezzo di una gather su tipo non contiguo.

La gather è cronometrata dopo una barriera, per la ragione discussa sopra:

```
MPI_Barrier(MPI_COMM_WORLD);      // park the straggler wait outside the timing
const double t0 = MPI_Wtime();     // of the gather (but inside T(p))
MPI_Gatherv(local.data(), my_rows * width, MPI_INT, recvbuf,
             px_counts.data(), px_displs.data(), MPI_INT, 0, MPI_COMM_WORLD);
g_last_comm_seconds = MPI_Wtime() - t0;
```

Lo schema dinamico è invece l'unico che richiede un'architettura dedicata. Il protocollo è minimale: il comando è un intero (l'indice di riga), il risultato è `width+1` interi — l'indice di riga seguito dagli escape time di quella riga — cosicché il master archivi il risultato all'arrivo senza tenere traccia di chi detenga cosa. Il cuore è il ciclo di eventi del master:

```
while (active > 0) {
    MPI_Recv(result.data(), width + 1, MPI_INT, MPI_ANY_SOURCE, TAG_RESULT,
              MPI_COMM_WORLD, &status);      // whoever finishes first
    // file the row into the matrix, then feed that same worker
    if (next_row < height) {
        MPI_Send(&next_row, 1, MPI_INT, status.MPI_SOURCE, TAG_WORK, ...);
        ++next_row;
    } else {
        MPI_Send(&none, 1, MPI_INT, status.MPI_SOURCE, TAG_STOP, ...);
        --active;
    }
}
```

`MPI_ANY_SOURCE` è ciò che realizza il self-scheduling: non è il master a decidere chi lavora, ma il primo worker che si libera a farsi avanti. La terminazione è implicita: ogni worker attivo detiene esattamente una riga in sospenso, quindi `active` si annulla quando tutte le righe sono state ricevute. Non serve alcuna gather finale, perché la matrice si compone incrementalmente sul master. Vale la pena notare il contrasto con OpenMP: lì la coda era un dettaglio interno al runtime, invisibile nel codice utente; qui la stessa politica richiede un protocollo di messaggi esplicito, un master dedicato e una condizione di terminazione da dimostrare. È il costo di complessità, prima ancora che di tempo.

Strato driver. MPI richiede un sottile adattamento del driver condiviso, isolato dietro `#ifdef USE_MPI` così da lasciare invariate le build seriale e OpenMP: inizializzazione e chiusura di MPI, una barriera che allinea l'inizio della regione cronometrata su tutti i rank, e l'I/O confinato al rank 0, l'unico a detenere la matrice assemblata. Il tempo di comunicazione raggiunge il CSV attraverso l'accessore `kernel_comm_seconds()`, nullo per seriale e OpenMP, senza che il kernel calcoli metriche.

Configurazione della misura: un solo nodo, per vincolo. Il job gira su un **unico nodo**, per un vincolo dell’ambiente e non per scelta metodologica (sezione 1.1): una richiesta con `--nodes=2` viene rifiutata da SLURM al momento della sottomissione. Il paradigma nato per la memoria distribuita viene dunque misurato, per forza di cose, su memoria condivisa.

L’allocazione è di 33 task su un nodo, con i rank ancorati ai core e assegnati in ordine, riempiendo un socket prima di passare all’altro (`--map-by core --bind-to core`). È l’analogo MPI del `OMP_PROC_BIND=close` usato per OpenMP: senza, il runtime potrebbe distribuire i due soli rank di $p = 2$ su socket diversi, caricando i punti a basso p di una penalità inter-socket estranea alla decomposizione. Soltanto a $np = 33$ un rank risiede necessariamente sul secondo socket. Si noti che questo posizionamento $rank \rightarrow core$ è cosa distinta dalla decomposizione $riga \rightarrow rank$ che porta lo stesso nome “a blocchi”.

La conseguenza va dichiarata apertamente. Tutti i messaggi transitano per la memoria condivisa del nodo e non per la rete, dunque la `comm_fraction` misurata è un *limite inferiore ottimistico* rispetto a quanto pagherebbe un’esecuzione realmente distribuita: quantifica correttamente *quanta* comunicazione ciascuno schema richiede, ma ne sottostima il prezzo unitario. Il divario fra master-worker e schemi statici va perciò letto come ottimistico verso il primo. In cambio si ottiene che T_1 e $T_{||}(p)$ sono garantiti sullo stesso hardware per costruzione, e che le curve MPI e OpenMP diventano direttamente confrontabili nei valori assoluti, non solo nelle tendenze: la loro differenza riflette in larga misura il solo modello di programmazione.

Il lanciatore: mpirun e non srun. La versione di OpenMPI disponibile sul cluster non può essere lanciata direttamente da `srun`: con il modulo `amd/gcc-8.5.0/openmpi-4.1.6` ogni rank termina dentro `MPI_Init` con l’errore

```
OPAL ERROR: Unreachable in file ext3x_client.c at line 111
```

La causa è un disallineamento di versione nello strato di avvio: il client PMIx di OpenMPI 4.1.6 è costruito per PMIx 3.x e non dialoga con il server PMIx v5 esposto da SLURM. I rank sono quindi lanciati con `mpirun`, che legge l’allocazione dalle variabili d’ambiente di SLURM e avvia i processi con il PMIx interno a OpenMPI. Il posizionamento dei rank resta invariato (`--map-by core --bind-to core`), mentre la variabile `MPI_DECOMP` va inoltrata esplicitamente con `-x`, perché `mpirun` propaga di default le sole variabili `OMPI_*`. La baseline seriale, che non usa MPI, continua a essere lanciata con `srun`.

Restano valide le scelte già motivate per OpenMP e per l’ambiente di misura (sezione 1.1): T_1 misurato con il binario seriale vero sullo stesso nodo, compilazione dentro il job, kernel *bare* come riferimento di scaling.

5.4.4 Analisi dei risultati

Il confronto mette a sistema i tre schemi sullo stesso intervallo di unità di calcolo, a griglia e $N_{\max}$ fissati. La tabella 8 raccoglie i 15 punti misurati.⁶

⁶Job 34137 su `gnode01`, partizione `ulow`. Gli speedup sono riferiti alla baseline seriale misurata *dentro lo stesso job*, $T_1 = 0,72021$ s; lo sweep OpenMP usa analogamente la propria, $0,72563$ s. Le due differiscono dello 0,75%, che è il fondo di risoluzione di qualunque confronto *fra* paradigmi condotto sugli speedup: per questo il raffronto diretto con OpenMP, più avanti, è fatto sui tempi.

Schema	p	$S \leq p/\lambda_p$	$T_{ }(p)$ [s]	$S(p)$	$E(p)$	$e(p)$	comm. [%]
seriale	1	—	0,72021	—	—	—	—
block	2	2,00	0,36113	1,994	0,997	0,0029	0,13
block	4	2,13	0,33965	2,120	0,530	0,2955	0,08
block	8	3,41	0,21255	3,388	0,424	0,1944	0,14
block	16	6,17	0,11759	6,125	0,383	0,1075	0,31
block	32	12,02	0,06157	11,697	0,366	0,0560	1,20
cyclic	2	2,00	0,36252	1,987	0,993	0,0067	0,18
cyclic	4	4,00	0,18126	3,973	0,993	0,0022	0,25
cyclic	8	7,99	0,09096	7,918	0,990	0,0015	0,24
cyclic	16	15,97	0,04930	14,608	0,913	0,0064	3,37
cyclic	32	31,70	0,02411	29,871	0,933	0,0023	3,02
dynamic	2	2,00	0,36199	1,990	0,995	0,0053	0,43
dynamic	4	4,00	0,18154	3,967	0,992	0,0028	0,68
dynamic	8	8,00	0,09092	7,921	0,990	0,0014	0,86
dynamic	16	16,00	0,04622	15,583	0,974	0,0018	2,21
dynamic	32	32,00	0,02316	31,102	0,972	0,0009	2,54

Tabella 8: MPI: metriche per schema di decomposizione e numero di unità di calcolo. Griglia 1024^2 , $N_{\max} = 1000$, 33 rank su un unico nodo. Per lo schema dinamico p indica i worker; i processi allocati sono $p + 1$. La colonna $S \leq p/\lambda_p$ riporta il limite predetto a priori dal profilo di carico seriale (tabella 5) e non è una misura: è il termine di confronto contro cui va letto lo speedup osservato.

p	T OpenMP dynamic, 1 [s]	T MPI dynamic [s]	divario	comm. MPI [%]
2	0,36079	0,36199	+0,33%	0,43
4	0,18043	0,18154	+0,62%	0,68
8	0,09028	0,09092	+0,71%	0,86
16	0,04542	0,04622	+1,76%	2,21
32	0,02342	0,02316	−1,11%	2,54

Tabella 9: Confronto dei due scheduler dinamici a parità di politica (concessione di una riga): stessa strategia su modelli di memoria diversi. Il confronto è sui *tempi* e non sugli speedup, così da essere indipendente dalla baseline seriale usata dai due sweep. Divario positivo significa MPI più lento. Si ricordi che a p unità di calcolo MPI occupa $p + 1$ core, OpenMP p .

Il quadro d'insieme. A 32 unità di calcolo, sullo stesso hardware e sullo stesso carico, lo speedup va da 11,70 (block) a 31,10 (master-worker): un fattore 2,66 fra la regola peggiore e la migliore, che coincide entro lo 0,1% con lo sbilanciamento λ_{32} previsto per i blocchi dal solo profilo seriale. Il rapporto fra speedup misurato e limite previsto è compreso fra 0,972 e 0,997 in tredici punti su quindici; le due eccezioni sono il cyclic a $p = 16$ e $p = 32$ (0,915 e 0,942), discusse più avanti. Il confronto punto per punto, esteso a OpenMP, è nella sezione 5.5.

Il block è limitato dal proprio λ_p . Il dato più vistoso della tabella è il passaggio da $p = 2$ a $p = 4$: raddoppiando i processi il tempo scende da 0,36113 s a 0,33965 s, cioè del 6%. Non è un'anomalia di misura: è λ_p che passa da 1,0000 a 1,8798 e si mangia per intero il raddoppio. Il punto $p = 2$ merita una nota, perché è ingannevole: la sua efficienza quasi perfetta (0,997) non dice che il block scali bene a p piccolo, ma che la viewport è simmetrica rispetto all'asse reale, sicché le due metà dell'immagine contengono esattamente lo stesso numero di iterazioni (129 827 745 ciascuna) e $\lambda_2 = 1$ per una proprietà geometrica dell'insieme, non della decomposizione.

Al crescere di p il λ_p dei blocchi continua a crescere, fino al valore intrinseco $\lambda = 2,76$ del caso limite $p = H$. L'efficienza tende perciò a stabilizzarsi attorno a $1/\lambda_p$, pari a 0,376 a $p = 32$ contro lo 0,366 misurato, mentre lo speedup continua a crescere: già a $p = 32$, però, circa il 63% del tempo di calcolo aggregato va perso in attesa.

Lettura di Karp-Flatt. Il block è il caso di scuola in cui la metrica segnala correttamente un problema attribuendolo però alla causa sbagliata. Se il deficit fosse una frazione seriale alla Amdahl, $e(p)$ sarebbe costante; qui invece decresce di un fattore cinque, da 0,2955 a 0,0560, e segue la forma $e(p) \approx (\lambda_p - 1)/(p - 1)$ già ricavata per lo static di OpenMP (sezione 5.3.4), che realizza la stessa regola: i valori predetti, 0,2933, 0,1927, 0,1062 e 0,0536 per $p = 4, 8, 16, 32$, riproducono i misurati entro l'1% fino a $p = 16$; a $p = 32$ lo scarto sale al 4,5%, nello stesso punto in cui lo speedup si allontana dal limite. La “frazione seriale sperimentale” non misura codice seriale, ma sbilanciamento.

Block e cyclic: bilanciamento a costo zero. Il primo confronto isola il puro effetto del bilanciamento: la comunicazione è la stessa unica gather, dunque ogni differenza di tempo è attribuibile alla sola distribuzione delle righe. Fino a $p = 8$ il cyclic è indistinguibile dall'ideale — $E = 0,993, 0,993, 0,990$ a 2, 4 e 8 processi — mentre il block, sulla stessa gather e sullo stesso carico, si ferma a 0,997, 0,530 e 0,424. A $p = 8$ il rapporto fra i due tempi è 2,34, contro il 2,35 predetto dal rapporto dei rispettivi λ_p : l'intero divario è distribuzione delle righe. La `comm_fraction` chiude l'argomento: 0,18%, 0,25%, 0,24% per il cyclic contro 0,13%, 0,08%, 0,14% per il block — la stessa cosa entro il rumore di misura. Non c'è stato alcun dialogo aggiuntivo: solo la medesima gather finale.

Il master-worker e l'effetto coda. Il master-worker firma il miglior punto assoluto dell'intera campagna CPU: $S = 31,10$ su 32 unità di calcolo, $E = 0,972, 89,7$ GFLOP/s contro i 2,88 della baseline seriale. Il residuo del 2,8% ha una spiegazione strutturale: è l'effetto coda. L'ultima riga consegnata a un worker può essere la più pesante della griglia, e con $\lambda_{\text{riga}} = 2,76$ la perdita di coda vale al più $(\lambda_{\text{riga}} - 1)p/H$, cioè il 5,5% a $p = 32$: il 2,8% misurato sta dentro quel margine. Sul fronte del coordinamento, il master resta lontano dalla saturazione: a $p = 32$ un worker spende circa 18 μs di round-trip per riga, contro 0,70 ms di calcolo medio: circa quaranta volte meno.

Va ribadita la contabilità: a 32 worker lo schema alloca 33 core, uno dei quali non calcola. L'efficienza per unità di calcolo è 0,972, quella per core allocato $31,10/33 = 0,942$. Entrambi i

numeri sono legittimi, ma vanno dichiarati insieme: a parità di core *allocati*, e non di unità di calcolo, il vantaggio del master-worker sul cyclic si assottiglia sensibilmente.

Cyclic e master-worker: quando il coordinamento adattivo comincia a valere. Fino a $p = 8$ i due schemi sono indistinguibili: 0,09096 s contro 0,09092 s, uno scarto dello 0,04%, un ordine di grandezza sotto la dispersione delle ripetizioni; a $p = 4$ il cyclic è addirittura più veloce dello 0,15%. Su questo intervallo la conclusione forte regge per intero: il self-scheduling non ripaga la propria complessità, perché una decomposizione statica accorta bilancia già quanto basta. A $p = 16$ e $p = 32$, però, il master-worker impiega il 6,3% e il 4,0% di tempo in meno del cyclic.

Il deficit del cyclic ad alto p non è imputabile né a λ_p , che è quasi 1, né alla gather, che è la stessa del block. Il candidato è il riordino di de-interlacciamento discusso sopra: una copia seriale su rank 0 dell'intera matrice, 4 MiB letti e 4 MiB scritti, di costo indipendente da p e stimabile attorno al millisecondo. A $p = 32$ il divario dal master-worker è di 0,95 ms, coerente con la stima; a $p = 16$ è invece di 3,1 ms, tre volte tanto, e un costo fisso non basta a spiegarlo. Poiché $p = 16$ è anche il punto con la dispersione più alta, l'attribuzione resta un'ipotesi: la verifica diretta, cronometrando il riordino o eliminandolo con il tipo derivato `MPI_Type_vector`, rientra fra gli sviluppi del lavoro.

La comunicazione non è il collo di bottiglia. In termini assoluti la comunicazione è irrilevante in tutta la campagna: fra 0,22 e 1,66 ms su tempi che vanno da 23 ms a 361 ms, mai oltre il 3,4% di $T_{\parallel}(p)$. Il master-worker scambia $2H = 2048$ messaggi per ripetizione contro l'unica gather collettiva degli schemi statici, e nonostante ciò resta sotto il 2,6%: alla grana di una riga (0,70 ms di calcolo contro 18 μ s di protocollo) il messaggio non è il fattore limitante.

Tre cautele accompagnano però questi numeri. Primo, `comm_fraction` non è la stessa grandezza nei tre schemi: negli statici misura la sola `MPI_Gatherv`, nel dinamico include anche l'attesa del comando successivo; confrontare il 2,54% del dinamico con l'1,20% del block come se fossero omogenei è scorretto. Secondo, per il block quasi tutto il tempo perso non è in `comm`: a $p = 32$ lo schema ha 1,20% di comunicazione e 63% di inattività parcheggiata nella barriera, conteggiata in $T_{\parallel}(p)$ ma non in `comm_fraction`; letta da sola, la colonna suggerirebbe che il block è efficiente. Terzo, questi valori sono su memoria condivisa: con la rete di mezzo il master-worker, *latency-bound* su 2048 messaggi piccoli, degraderebbe assai più del cyclic, *bandwidth-bound* su una singola collettiva — al punto che la classifica potrebbe invertirsi. È la conclusione più esposta al limite del nodo singolo.

Scheduler dinamici a confronto: il costo del modello di memoria. Il terzo confronto, in tabella 9, mette a fronte i due scheduler dinamici sulla stessa politica: a concessione unitaria da entrambi i lati, il loro divario quantifica il costo del solo cambio di modello di memoria. Fino a $p = 16$ MPI paga, e paga poco: dallo 0,33% all'1,76%. È il prezzo del protocollo rispetto a un prelievo da contatore condiviso, e la sua crescita con p è quella attesa da un coordinatore centralizzato.

A $p = 32$ il segno si inverte: MPI è più veloce dell'1,11% (0,02316 s contro 0,02342 s). Da un lato, `dynamic,1` di OpenMP mostra proprio a 32 thread il suo unico scostamento apprezzabile dall'ideale (E da 0,999 a 0,968): 1024 prelievi da un unico contatore condiviso fra 32 thread cominciano a costare contesa. Dall'altro, una possibile spiegazione è che il master MPI serializzi le richieste su un core proprio, senza contendere cache line ai worker. Alla scala massima misurata, il costo della coda in memoria condivisa supera quello del passaggio di messaggi. Resta la contabilità: quel vantaggio si ottiene occupando un 33° core, e a parità di core allocati l'ordine si ristabilisce a favore di OpenMP di circa il 2%.

Correttezza e qualità della misura. Tutte e sedici le righe del file di run condividono il medesimo checksum (9203734293765563287), uguale a quello della baseline seriale: le tre decomposizioni riproducono la matrice degli escape time bit per bit, su conteggi di rank che dividono e non dividono l'altezza, e nessuno schema ha alterato il risultato. Le sei baseline seriali a 1024^2 e $N_{\max} = 1000$ registrate nel dataset, una per ciascun job, valgono fra 0,71990 e 0,72563 s e concordano entro lo 0,8%: è una verifica indipendente della riproducibilità del nodo. La dispersione fra ripetizioni, misurata come scarto della mediana dal minimo, resta sotto il 2% nella maggior parte dei punti, ma sale al 4,4% per il master-worker a $p = 32$ e al 5,1% e 6,6% per il cyclic a $p = 16$ e $p = 32$, fra i punti più brevi e quindi più esposti alla contesa su `gnode01`. L'efficienza non monotona del cyclic (0,913 a $p = 16$ contro 0,933 a $p = 32$) non ha spiegazione strutturale ed è il punto da rimisurare prima di costruirvi sopra un'affermazione.

5.5 Validazione a posteriori delle previsioni del baseline

Con entrambe le campagne CPU disponibili, i limiti della sezione 5.2 possono essere confrontati con gli speedup misurati. La tabella 10 mette a fronte, per ciascuna delle tre regole, il limite predetto e le misure dei meccanismi che la realizzano: venticinque punti di misura contro quindici valori predetti; la figura 7 ne offre la lettura visiva. Alla stessa lettura appartiene un caso che la tabella non contempla: uno scheduling dinamico a chunk fissi di c righe eredita il limite della decomposizione a blocchi con H/c blocchi, come mostra `dynamic, 64` nella sezione 5.3.4.

Regola	p	λ_p	$S \leq p/\lambda_p$	S OpenMP	scarto	S MPI	scarto
<i>blocchi</i> — OpenMP static, MPI block							
	2	1,0000	2,000	2,011	+0,6%	1,994	−0,3%
	4	1,8798	2,128	2,132	+0,2%	2,120	−0,4%
	8	2,3489	3,406	3,410	+0,1%	3,388	−0,5%
	16	2,5937	6,169	6,176	+0,1%	6,125	−0,7%
	32	2,6619	12,022	11,999	−0,2%	11,697	−2,7%
<i>ciclica</i> — solo MPI cyclic							
	2	1,0000	2,000	—	—	1,987	−0,7%
	4	1,0001	4,000	—	—	3,973	−0,7%
	8	1,0011	7,991	—	—	7,918	−0,9%
	16	1,0017	15,972	—	—	14,608	−8,5%
	32	1,0095	31,698	—	—	29,871	−5,8%
<i>dinamica</i> — OpenMP dynamic, 1, MPI master-worker							
	2	1,0000	2,000	2,011	+0,6%	1,990	−0,5%
	4	1,0000	4,000	4,022	+0,5%	3,967	−0,8%
	8	1,0000	8,000	8,037	+0,5%	7,921	−1,0%
	16	1,0000	16,000	15,977	−0,1%	15,583	−2,6%
	32	1,0000	32,000	30,988	−3,2%	31,102	−2,8%

Tabella 10: Validazione a posteriori: speedup misurato contro il limite p/λ_p predetto dal solo profilo di carico seriale, per ciascuna delle tre regole di assegnamento e per ciascun meccanismo che la realizza. Griglia 1024^2 , $N_{\max} = 1000$, kernel *bare*. Scarto positivo significa misura sopra il limite predetto. Ogni paradigma è riferito alla baseline seriale del proprio job (0,72563 s per OpenMP, 0,72021 s per MPI): le due differiscono dello 0,75%, che è quindi il fondo di risoluzione della colonna “scarto” e la ragione per cui gli scostamenti sotto l'1% non vanno confrontati fra loro nel segno.

Lo statico satura il limite predetto. Per la regola a blocchi l'accordo con `schedule(static)` è entro lo 0,6% su tutti e cinque i punti. La decomposizione statica non lascia quindi margine di ottimizzazione implementativa, perché tocca già il proprio limite teorico: la perdita del 62% di efficienza a 32 thread non è imputabile a overhead di OpenMP, a cache o a sincronizzazione, che insieme valgono meno dello 0,6%, ma unicamente al fatto che le righe costose dell'insieme sono contigue e finiscono nello stesso blocco. L'unica via per superarla è cambiare regola di assegnamento.

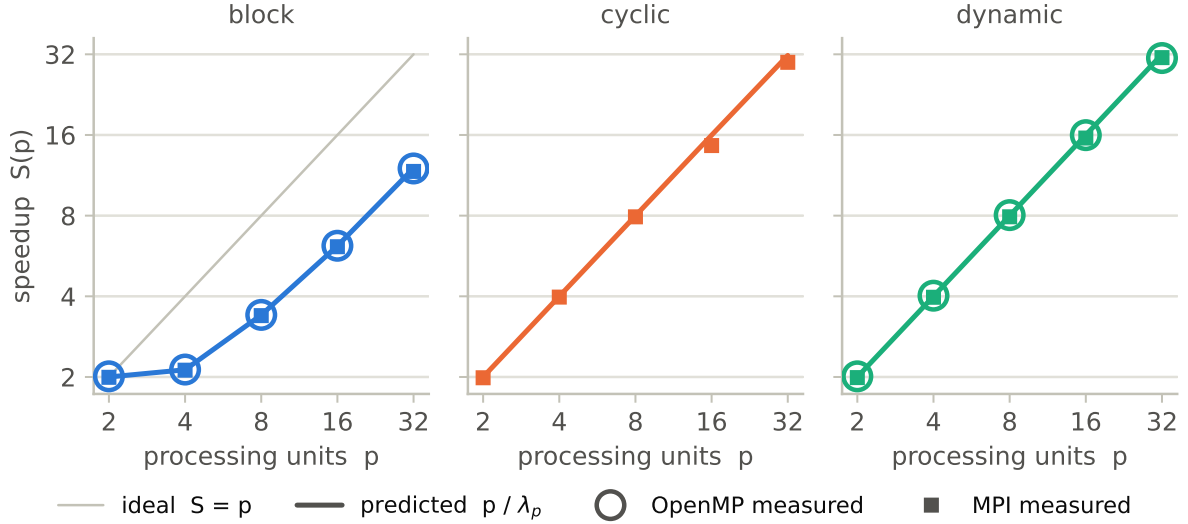


Figura 7: Speedup misurato contro il limite p/λ_p previsto dal profilo di carico seriale, un pannello per regola di assegnamento. La linea è il limite previsto, gli anelli le misure OpenMP, i quadrati le misure MPI; la retta grigia è l'ideale $S = p$. Assi logaritmici in base 2; i dati sono quelli della tabella 10.

La previsione dipende dalla regola, non dal paradigma. Il risultato che la tabella rende visibile è che la previsione dipende dalla *regola* e non dal meccanismo. Le due realizzazioni della regola a blocchi — una clausola `schedule(static)` in memoria condivisa e una decomposizione esplicita con gather collettiva in memoria distribuita — cadono sullo stesso limite predetto entro pochi decimi di punto fino a $p = 16$, pur non avendo in comune né il modello di memoria né il lanciatore. Lo stesso vale per la regola dinamica, dove uno scheduler di runtime e un protocollo master-worker esplicito arrivano a 30,99 e 31,10 su un limite di 32,00. Poiché λ_p è calcolato dal solo $\{w_r\}$ seriale, questa coincidenza è la conferma più solida del modello predittivo: la stessa riga di previsione governa due implementazioni indipendenti.

La regola ciclica, che lo sweep OpenMP non esercitava e che restava priva di verifica, è ora confermata dove il modello è più esposto: prevede $\lambda_p \approx 1$ e quindi speedup quasi ideale, e il cyclic lo raggiunge entro l'1% fino a $p = 8$. È la conferma che λ_p non si limita a descrivere un fallimento — come faceva per i blocchi — ma predice anche il successo di una regola alternativa.

Dove il modello smette di predire, e perché. Tre scostamenti escono dal margine, e ciascuno indica un termine di costo che il modello compute-only (8) per costruzione non contiene.

Il primo è la regola dinamica ai valori più alti di p : a $p = 32$ entrambi i meccanismi si fermano attorno al 97% del limite (−3,2% OpenMP, −2,8% MPI). Il modello del makespan tratta il lavoro come frazionabile a piacere, mentre l'unità indivisibile è la riga: l'ultima riga assegnata a un worker può essere la più pesante, e questo *effetto coda* riduce il limite di al più $(\lambda_{\text{riga}} - 1)p/H$, cioè il 2,8% a $p = 16$ e il 5,5% a $p = 32$. Tutte le misure dinamiche restano dentro questo margine. Per OpenMP vi contribuisce anche la contesa sulla coda condivisa (sezione 5.3.4), e i dati non permettono di separare i due termini.

Il secondo è *cyclic* a $p = 16$ e $p = 32$ (−8,5% e −5,8%), l'unico caso in cui lo scarto è grande e non ha spiegazione dentro il modello: come discusso nella sezione 5.4.4, la causa candidata è il riordino seriale di de-interlacciamento su rank 0, un costo di implementazione indipendente da p e quindi invisibile a una previsione fondata sul solo profilo di carico, che però non spiega da solo lo scarto a $p = 16$. Quel punto è anche il più rumoroso della campagna e va rimisurato.

Il terzo è **block MPI** a $p = 32$ ($-2,7\%$, contro il $-0,2\%$ della realizzazione OpenMP della stessa regola). La gather ne spiega poco meno della metà, pesando l' $1,2\%$ del tempo; il resto non è attribuibile con le misure disponibili. Questi scostamenti corrispondono ai termini che il modello non include — granularità finita, costi di implementazione, comunicazione — e nessuno supera l' $8,5\%$ sui venticinque punti di misura.

Conseguenza interpretativa. Che le regole statiche saturino il proprio limite entro frazioni di punto percentuale ha un effetto sulla lettura di tutto il lavoro: gli scostamenti dallo speedup ideale qui osservati non originano da una frazione seriale nel senso di Amdahl — che nel codice è trascurabile — ma dalla distribuzione del lavoro. Vanno perciò letti attraverso λ_p più che attraverso la metrica di Karp–Flatt, che pure li segnala correttamente ma li attribuisce a una causa che questo codice non ha: le forme chiuse di $e(p)$ ricavate per lo **static** di OpenMP (sezione 5.3.4) e per il **block MPI** (sezione 5.4.4) mostrano che la “frazione seriale sperimentale” non è che λ_p riscritto in un’altra variabile. È il criterio con cui vanno lette tutte le colonne $e(p)$ di questo capitolo.

5.6 CUDA: acceleratore grafico

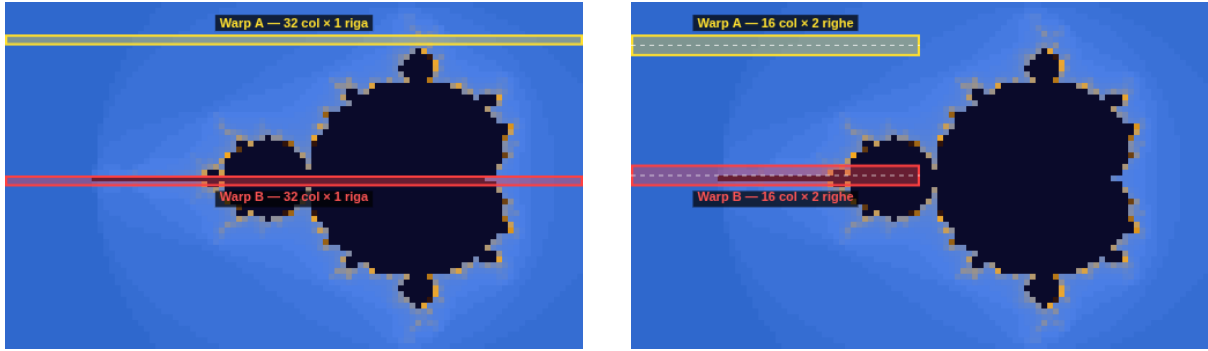
Con CUDA cambia il modello di esecuzione. Il pixel resta l’unità indipendente (sezione 5), ma non viene più raggruppato in righe distribuite fra poche unità di calcolo: ciascun pixel è affidato a uno dei migliaia di thread della GPU.

L’hardware non esegue i thread singolarmente, ma li organizza in tre livelli. La *griglia* è l’insieme di tutti i *blocchi*, ciascuno dei quali viene assegnato per intero a uno Streaming Multiprocessor (SM) [8]. All’interno del blocco i thread sono numerati con la formula $\text{id} = \text{threadIdx.x} + \text{threadIdx.y} \cdot \text{blockDim.x}$, e l’SM li esegue in gruppi di 32 id consecutivi — i *warp* [9]. Ogni warp avanza con un unico program counter: i suoi 32 thread eseguono la stessa istruzione nello stesso ciclo secondo il modello SIMT (Single Instruction, Multiple Threads). Il warp scheduler non conosce la struttura 2D che si è dichiarata: vede solo id lineari e ne prende 32 alla volta.

Questa linearizzazione è il punto critico per il problema in esame. Con $\text{blockDim.x} = 32$, i thread 0–31 hanno tutti $\text{threadIdx.y} = 0$, cioè appartengono alla stessa riga del blocco; il cambio di riga scatta solo all’id 32, che è già nel warp successivo. Con $\text{blockDim.x} = 16$, invece, il thread con $\text{id} = 16$ ha $\text{threadIdx.x} = 0$, $\text{threadIdx.y} = 1$: il cambio di riga cade a metà warp, e il warp 0 finisce per contenere i pixel delle colonne 0–15 della riga 0 e delle colonne 0–15 della riga 1. In generale, per blockDim.x divisore di 32, ogni warp copre $32/\text{blockDim.x}$ righe.

Quando un warp copre più righe si pongono due questioni distinte.

- **Coalescenza.** La matrice è memorizzata in ordine row-major: pixel della stessa riga sono contigui in memoria, e 32 scritture contigue possono essere servite in una sola transazione. Se il warp scrive su più righe gli indirizzi non sono più contigui e servono più transazioni, fino a 32 volte il traffico per gli stessi dati.
- **Divergenza.** Le 32 lane di un warp eseguono insieme, ma richiedono in generale un numero diverso di iterazioni: il warp termina solo quando termina la lane più lenta, e le lane già uscite consumano cicli con il risultato mascherato. Il costo del warp è quindi il massimo di $n_{i,j}$ sulle sue lane, invece della media, e la frazione di cicli sprecati è $1 - \bar{n}/\max n$. Un warp confinato in una riga non azzera la divergenza, perché una riga che attraversa il bordo contiene pixel interni ed esterni; un warp che riunisce righe diverse può però accostare situazioni ancora più eterogenee, per esempio pixel esterni in una riga e pixel di un’antenna nella riga successiva.



(a) $\text{blockDim.x} = 32$: ogni warp occupa esattamente una riga. Il warp A (giallo) cade in una zona esterna, dove tutti i pixel sfuggono in poche iterazioni; il warp B (rosso) attraversa il bordo frattale, dove convivono pixel interni ed esterni con escape time molto diversi. Questa varianza dipende dalla geometria del problema, non dalla scelta del blocco.

(b) $\text{blockDim.x} = 16$: ogni warp copre due righe (la linea tratteggiata segna il confine). Il warp A resta omogeneo, perché entrambe le righe cadono in zona esterna; nel warp B le due righe possono campionare strutture diverse del bordo, e la varianza degli escape time può crescere rispetto al caso a riga singola.

Figura 8: Mappatura dei warp sulla griglia degli escape time al variare di blockDim.x ; la risoluzione dell'immagine e l'estensione dei warp sono illustrative. In entrambe le figure sono evidenziati un warp in zona esterna (A, giallo) e uno sul bordo frattale (B, rosso). L'ipotesi illustrata è che riunire più righe in un warp possa aumentare la divergenza, oltre a spezzare la coalescenza; la sezione 5.6.4 ne quantifica l'effetto medio sull'intera immagine.

Come mostra la figura 8, la scelta $\text{blockDim.x} = 32$, o un suo multiplo, garantisce che i 32 id consecutivi di un warp stiano sempre sulla stessa riga: le scritture restano coalescenti, ed è esclusa la divergenza aggiuntiva dovuta all'accostamento di righe diverse. blockDim.y resta libero, perché determina quante righe gestisce il blocco ma non la forma dei warp. Per questo la scelta $\text{threadIdx.x} \rightarrow \text{colonna}$, $\text{threadIdx.y} \rightarrow \text{riga}$ non è una convenzione neutra. Si tratta tuttavia di un'ipotesi da verificare: un blocco compatto riunisce pixel vicini in entrambe le direzioni, e in media potrebbe produrre warp non meno omogenei di una riga. Le previsioni della sezione 5.6.4 e le misure della sezione 5.6.5 ne quantificano l'effetto.

La varianza per pixel degli escape time è la stessa irregolarità discussa fin dall'inizio del lavoro, che si manifesta come sbilanciamento del carico sulla CPU e come divergenza dei warp sulla GPU.

L'aritmetica del kernel è identica a quella seriale, in doppia precisione, e il build impone `--fmad=false`, l'analogo per nvcc di `-ffp-contract=off`: la contrazione in fused multiply-add cambierebbe l'ultimo bit e quindi ± 1 iterazione sui pixel di bordo, rendendo il checksum incomparabile con la baseline. La L40S dispone inoltre di tensor core, progettati per le moltiplicazioni fra matrici dense [8]: il kernel, scalare e per pixel, non ne fa uso per costruzione.

5.6.1 Design dello sweep

La domanda progettuale è la stessa delle sezioni precedenti: *come* mappare il lavoro sulle unità di calcolo, non *quanto*. Sulla GPU la variabile di controllo non è il numero di core — fisso per la scheda — ma la *geometria del blocco*, ovvero la forma $B_x \times B_y$ con cui i thread vengono organizzati, e che determina simultaneamente la struttura dei warp, la coalescenza degli accessi in memoria e l'occupancy degli SM. Lo sweep si articola in due esperimenti ortogonali, ciascuno con la propria domanda.

Sweep A — geometria del blocco. Vengono fissati i parametri del problema (1024^2 , $N_{\max} = 1000$, come in tutti gli altri sweep), e si fa variare la sola forma del blocco su nove geometrie appartenenti a quattro gruppi distinti. L'esperimento risponde alla domanda: qual è la mappatura thread $\rightarrow$ pixel migliore, e perché? La maggior parte delle geometrie ha lo stesso numero totale di thread per blocco (256); ciò che varia è la loro distribuzione su righe e colonne, che determina la struttura dei warp e quindi coalescenza e divergenza. Le due eccezioni, 512×1 e 128×1 , servono a isolare l'effetto del numero di thread a geometria del warp fissa.

- **Warp orizzontali puri** (256×1 , 512×1 , 128×1): `blockDim.x` multiplo di 32, scritture coalescenti. Variano la taglia del blocco, e con essa potenzialmente l'occupancy, a coalescenza e divergenza fisse: è il gruppo di riferimento.
- **Quadrati 2D coalescenti** (128×2 , 64×4 , 32×8): con `blockDim.x` multiplo di 32, ogni warp prende 32 id consecutivi che stanno tutti sulla stessa riga. Quindi la coalescenza è intatta e la divergenza è la stessa dei warp orizzontali puri. La differenza rispetto al primo gruppo è solo `blockDim.y`: cambia quanti warp risiedono in un SM, e con essi potenzialmente l'occupancy. Isola l'effetto di avere più righe per blocco.
- **Coalescenza violata** (16×16 , 8×32): `blockDim.x` < 32 , ogni warp ripiega su più righe, spezzando gli accessi in memoria in più transazioni. Questo gruppo mette in luce un aspetto interessante dello sweep: un quadrato compatto come 16×16 raggruppa pixel più vicini fra loro e potrebbe avere divergenza *inferiore* a quella dei warp orizzontali (campionando una regione più stretta del piano), ma a prezzo di coalescenza violata. Divergenza e coalescenza tirano in direzioni opposte: non esiste una scelta vincente a priori, e solo il tempo di esecuzione può stabilire quale effetto prevalga.
- **Warp verticale** (1×256): la configurazione prevede 32 pixel della stessa colonna, tali che gli indirizzi sono distanti $W \cdot 4$ byte l'uno dall'altro. Si tratta pertanto del caso peggiore per la coalescenza. La divergenza, però, è quasi identica a quella dei warp orizzontali: non c'è nessuna direzione privilegiata in cui il problema sia "più omogeneo" e, di conseguenza, la varianza di n tra due pixel adiacenti in orizzontale è comparabile a quella tra due pixel adiacenti in verticale. Questo rende 256×1 contro 1×256 l'esperimento più pulito dello sweep: divergenza simile, coalescenza opposta. La differenza di tempo misurata è quindi imputabile principalmente alla penalità di coalescenza.

Vale la pena chiarire perché 1×256 non è l'analogo GPU della decomposizione ciclica MPI. Nel ciclico MPI il rank r riceveva righe sparse per l'intera immagine, mescolando righe leggere e pesanti; l'analogo GPU sarebbe un warp i cui 32 pixel sono prelevati da righe distanti anziché adiacenti. Quella configurazione avrebbe divergenza massima e accessi non coalescenti, e sarebbe il caso peggiore per entrambe le leve; non è realizzabile con una forma di blocco, ma il suo costo si può stimare dalla matrice degli escape time (sezione 5.6.4). La 1×256 , invece, rompe la coalescenza ma lascia pressoché intatta la divergenza, ed è per questo un controllo utile.

Il risultato atteso dallo Sweep A è la classifica delle nove geometrie per $T_{\min}$, insieme alla verifica di quanto le grandezze calcolabili dalla sola matrice degli escape time (sezione 5.6.4) ne anticipino l'ordinamento: è l'analogo di λ_p , che sul lato CPU prevedeva lo speedup (sezione 5.2).

Sweep B — saturazione e trasferimento. Viene fissata la forma del blocco 256×1 , riferimento a warp orizzontali, e si fanno variare i valori dei parametri del problema. Sulla GPU non è infatti il numero di core a variare, bensì il loro *utilizzo*. La scalabilità della GPU non è quindi una curva su p come per OpenMP e MPI, ma è una curva di *saturazione*: al crescere della taglia del problema cresce il numero di warp lanciati, e con essi l'occupazione degli SM, e le prestazioni in GFLOP/s, fino al punto in cui aggiungere pixel non produce più guadagno proporzionale. Oltre quella soglia la scheda è sfruttata appieno. Lo sweep si compone di due esperimenti.

B1 — saturazione: risoluzione $512^2 \rightarrow 1024^2 \rightarrow 2048^2 \rightarrow 4096^2$ a $N_{\max} = 1000$ fisso. Al crescere della risoluzione aumenta il numero di warp lanciati e con essi l'occupazione degli SM: la curva di throughput effettivo in GFLOP/s dovrebbe salire e appiattirsi verso il picco. La domanda è *da quale taglia* la scheda è sfruttata appieno, e quale fattore la penalizza sotto quella soglia: overhead di lancio del kernel, trasferimento incompressibile o SM non saturi.

B2 – trasferimento: $N_{\max} \in \{500, 1000, 5000\}$ a risoluzione 2048^2 fissa. A pixel fissi i byte da copiare sono costanti, mentre il calcolo cresce con $N_{\max}$: ci si attende che la frazione $\text{transfer_time}/T$ cali, mostrando l’ammortamento del costo di copia, i cui contributi sono descritti nella sezione 5.6.2, al crescere del lavoro. È il termine “alla Amdahl” del percorso $\text{host} \leftrightarrow \text{device}$, e la curva B2 lo quantifica.

Ogni punto di B lancia il seriale e la GPU sullo stesso nodo nella stessa esecuzione, così $S = T(1)/T_{\text{gpu}}$ è same-hardware per costruzione: un core CPU contro una GPU, con la baseline seriale rigenerata a ogni confronto. Il punto a 1024^2 e $N_{\max} = 1000$ ripete la configurazione 256×1 dello Sweep A con lo stesso numero di ripetizioni, e fa quindi da controllo fra i due job.

5.6.2 Metriche specifiche

Sulla GPU il numero di core è fissato dalla scheda e non è una variabile controllabile: al crescere del problema cresce la loro occupazione, non il loro numero. Il parametro p è quindi degenere, e con esso efficienza e metrica di Karp–Flatt, che presuppongono un numero variabile di unità di calcolo. Lo speedup è definito come $S = T_1/T_{\text{gpu}}$, dove T_1 è il tempo della baseline seriale su un core della CPU dello stesso nodo: è il confronto fra due implementazioni dello stesso problema, non fra configurazioni a p variabile.

Tre grandezze sono proprie di questo paradigma.

La **divergenza** è misurata tramite un proxy deterministico calcolato dalla matrice degli escape time: per ogni gruppo di 32 pixel contigui secondo la mappatura del kernel, si calcola la varianza dei valori $n_{i,j}$; la media di queste varianze su tutti i gruppi è il proxy. Poiché l’unico branch del kernel è il ciclo di escape, la divergenza dipende soltanto dagli escape time dei pixel riuniti nel warp, ed è quindi interamente calcolabile dalla matrice. Questo approccio è preferibile alla misura strumentata con un profiler per tre ragioni: è deterministico e privo di rumore, non richiede permessi sui performance counter (spesso non disponibili sui cluster), e non perturba i tempi di esecuzione.

L’**occupancy** è teorica e si ottiene dall’API runtime. In linea di principio il suo valore dipende dal numero di registri che il kernel impiega per thread, riportato da `-Xptxas -v`: il kernel mantiene attive più variabili in doppia precisione ($z_{\text{real}}, z_{\text{imag}}$, i rispettivi quadrati, c_{real} e c_{imag}), e registri insufficienti limiterebbero i warp che possono risiedere contemporaneamente su un SM. Un’occupancy elevata non implica tuttavia prestazioni migliori: serve a mascherare le latenze di memoria, mentre il kernel è limitato dal calcolo, con fino a $8 N_{\max}$ FLOP per ogni scrittura da 4 byte. La leva attesa come primaria è quindi la divergenza.

Il **tempo di trasferimento** $\text{device} \rightarrow \text{host}$ è misurato con CUDA event attorno alla copia della matrice risultante (4 byte per pixel). Non esiste traffico $\text{host} \rightarrow \text{device}$: gli input del kernel sono scalari passati come argomenti al lancio, senza alcuna `cudaMemcpy`. Poiché il costo di copia è fisso in byte mentre il calcolo cresce con $N_{\max}$, la grandezza di interesse è la frazione $\text{transfer_time}/T$, che è attesa calare all’aumentare di $N_{\max}$: lo Sweep B2 quantifica questo ammortamento.

Che cosa comprende il tempo di trasferimento. La copia della matrice dalla GPU alla memoria del processo non è un’unica operazione: il tempo misurato somma tre contributi,

$$T_{\text{transfer}} = T_{\text{PCIe}} + T_{\text{socket}} + T_{\text{host}}.$$

- T_{PCIe} è il trasferimento sul bus della scheda, un collegamento PCIe 4.0 x16 con una banda nominale di circa 31,5 GB/s per direzione;
- T_{socket} è l’eventuale attraversamento dell’interconnessione fra i due socket del nodo, nullo quando il processo gira sul socket a cui è collegata la GPU;

- T_{host} è la copia in memoria host: la matrice risiede in memoria ordinaria, che la GPU non può scrivere direttamente, per cui il driver trasferisce i dati in un buffer intermedio e li copia poi nella destinazione.

Il termine centrale dipende da una scelta che non è sotto il controllo del job. Il nodo è dual-socket (sezione 1.1), e ciascuna delle otto GPU è collegata a uno dei due socket; in assenza di richieste esplicite, SLURM assegna la GPU e il core del processo in base alla disponibilità del momento. Da un job all'altro il processo può quindi trovarsi sullo stesso socket della GPU, il caso locale, oppure sull'altro, il caso cross-NUMA (figura 9). Poiché `nvidia-smi` non riporta l'affinità NUMA della scheda, il job dello Sweep B la ricava da `sysfs` e registra nel log sia il socket del processo sia i core locali alla GPU (`mandelbrot/job_sbatch/mandel_cuda_scaling.sh`). Ogni valore di `transfer_time` viene così letto rispetto al posizionamento effettivo del proprio job, e non rispetto a uno presunto.

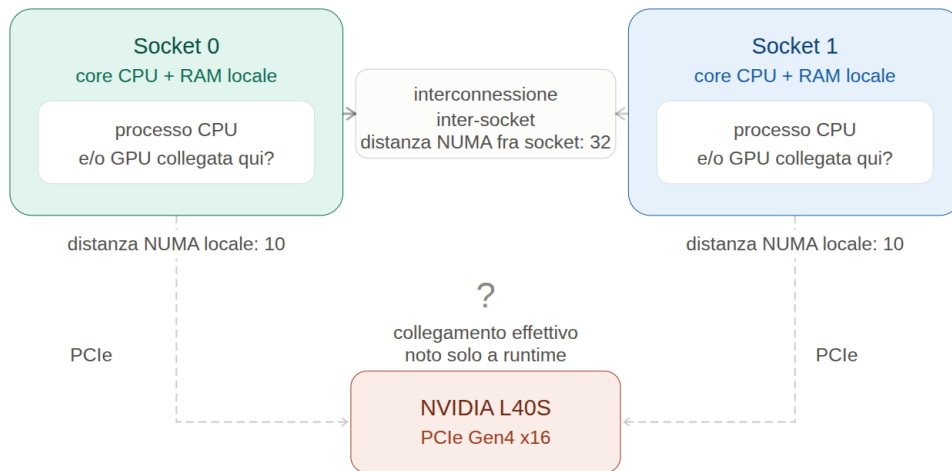


Figura 9: Topologia NUMA di gnode01. Ciascun socket dispone dei propri core e della propria memoria locale; i due socket sono collegati da un'interconnessione, per la quale il firmware dichiara una distanza NUMA di 32 contro 10 dell'accesso locale, un valore relativo e non una misura di latenza. La GPU è collegata tramite PCIe a uno dei due socket, e il processo può essere eseguito sullo stesso socket o sull'altro: nel secondo caso la copia della matrice attraversa anche l'interconnessione.

5.6.3 Frammenti di codice rilevanti

Nei due paradigmi precedenti il kernel seriale restava riconoscibile: OpenMP vi anteponeva una direttiva, MPI lo racchiudeva in un protocollo di messaggi. Con CUDA scompaiono invece i due cicli annidati sui pixel: ogni thread esegue una sola volta il corpo del ciclo e deve ricavare autonomamente il pixel di propria competenza. La ricorrenza di escape è identica a quella seriale e non viene riportata; il codice completo è in `mandelbrot/src/mandelbrot_cuda.cu`.

```
__global__ void mandelbrot_kernel(int* escape_counts, int width, int rows,
                                Viewport view, double dx, double dy,
                                int max_iter) {
    const int col = blockIdx.x * blockDim.x + threadIdx.x;
    const int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (col >= width || row >= rows) return;    // lanes beyond the image

    // c_real, c_imag: centre of pixel (row, col), as in the serial kernel
    escape_counts[row * width + col] = escape_time(c_real, c_imag, max_iter);
}
```

Le variabili `col` e `row` realizzano l'intera assegnazione del lavoro. Ogni thread dispone di tre terne predefinite: `threadIdx`, la posizione del thread all'interno del proprio blocco, con componenti comprese fra 0 e `blockDim-1`; `blockIdx`, la posizione del blocco all'interno della griglia; `blockDim`,

le dimensioni del blocco. Lungo ciascun asse il prodotto `blockIdx * blockDim` è il numero di thread contenuti nei blocchi che precedono quello corrente, e coincide quindi con la coordinata del primo pixel del blocco; sommandovi `threadIdx` si ottiene la coordinata globale del thread. `col` è pertanto l'indice di colonna del pixel e `row` il suo indice di riga (`rows` coincide con l'altezza dell'immagine quando la simmetria non è attiva): la coppia individua l'unica cella della matrice degli escape time che il thread calcola e scrive. L'associazione dell'asse x alle colonne è la scelta motivata nella sezione 5.6. Come l'indice di riga globale dei rank MPI (sezione 5.4.3), il compito non viene comunicato al thread ma ricavato dalla sua posizione; a differenza di MPI, l'unità assegnata non è una riga ma un singolo pixel.

La guardia successiva è necessaria perché la griglia è dimensionata per eccesso (frammento seguente): quando le dimensioni dell'immagine non sono multiple di quelle del blocco, i blocchi dell'ultima riga o colonna della griglia si estendono oltre l'immagine, e i thread corrispondenti devono terminare senza scrivere. Si osservi infine che nessuna istruzione del kernel dipende dalla forma del blocco: le nove geometrie dello Sweep A eseguono lo stesso codice e differiscono soltanto per l'insieme di pixel che condividono un warp.

```
static const dim3 block = block_shape_from_env();          // MANDEL_BLOCK=BxBy
static const double occupancy = launch_occupancy(block);
const dim3 grid((width + block.x - 1) / block.x,          // ceiling division
               (computed_rows + block.y - 1) / block.y);
mandelbrot_kernel<<<grid, block>>>(d_escape_counts, width, computed_rows, ...);
```

La forma del blocco è letta a esecuzione dalla variabile d'ambiente `MANDEL_BLOCK`, analogamente a quanto avviene con la variabile `OMP_SCHEDULE` per OpenMP e con la variabile `MPI_DECOMP` per MPI: un solo eseguibile copre così tutte le geometrie dello sweep. Forma del blocco e `occupancy`, che dipende soltanto da essa, sono determinate una volta per processo (`static`); la griglia è invece ricalcolata a ogni chiamata, poiché dipende dalla risoluzione, che lo Sweep B fa variare.

Il frammento più rilevante per l'analisi non appartiene al kernel, ma al codice condiviso delle metriche (`mandelbrot/src/metrics.cpp`), che calcola divergenza e spreco sulla matrice degli escape time. Per ogni blocco della griglia, di indici `bx` e `by`, i thread vengono raggruppati in warp di 32 id consecutivi e ciascun id viene ricondotto al proprio pixel:

```
for (long long start = 0; start < threads_per_block; start += warp_size) {
    lane.clear();                                // lanes of one warp
    const long long end = std::min(start + warp_size, threads_per_block);
    for (long long id = start; id < end; ++id) {
        const int col = bx * block_x + static_cast<int>(id % block_x);
        const int row = by * block_y + static_cast<int>(id / block_x);
        if (col < width && row < height)
            lane.push_back(img.data[row * width + col]);
    }
    // accumulate var(lane) and 1 - mean(lane) / max(lane)
}
```

Le espressioni di `col` e `row` sono quelle del kernel, con la posizione del thread nel blocco ricavata dall'id lineare invertendo la formula della sezione 5.6: `threadIdx.x = id mod blockDim.x` e `threadIdx.y = [id/blockDim.x]`. Il raggruppamento riproduce quindi esattamente quello operato dal warp scheduler, ed è per questo che divergenza e spreco sono previsioni disponibili prima dell'esecuzione del kernel. Le lane che cadono oltre l'immagine sono escluse, affinché entrambe le grandezze misurino la variabilità indotta dal frattale e non gli effetti di bordo della suddivisione in blocchi. Coerentemente con il contratto delle metriche, il calcolo risiede nel driver condiviso e non nel kernel.

Due ulteriori accorgimenti allineano la regione misurata al contratto comune agli altri paradigmi (allocazione inclusa, I/O escluso):

```
const ContextInit g_context_init;    // ctor: cudaFree(nullptr) -> context

cudaEventRecord(before_copy);        // queued behind the kernel, same stream
cudaMemcpy(img.data.data(), d_escape_counts, bytes, cudaMemcpyDeviceToHost);
cudaEventRecord(after_copy);         // elapsed(before, after) = pure D2H
```

Il primo riguarda il contesto CUDA, la cui creazione, eseguita alla prima chiamata all'API, richiede centinaia di millisecondi: l'oggetto statico `g_context_init` lo crea prima dell'avvio del cronometro, così che il costo non ricada sulla prima ripetizione. Il secondo riguarda il tempo di trasferimento: i due event sono accodati sullo stesso stream del kernel, che esegue le operazioni in ordine, quindi `before_copy` viene raggiunto solo al termine del kernel e l'intervallo misurato comprende la sola copia device→host.

Le grandezze proprie della GPU raggiungono il CSV attraverso accessori dichiarati nell'header comune (`kernel_occupancy()`, `kernel_transfer_seconds()` e le dimensioni del blocco), che i kernel CPU definiscono nulli: il medesimo driver produce così lo stesso schema di record per tutti i paradigmi.

5.6.4 Previsioni dalla matrice degli escape time

Come per le decomposizioni CPU (sezione 5.2), il costo delle geometrie del blocco può essere stimato prima di eseguire il kernel. La forma del blocco determina quali pixel condividono un warp, e questo raggruppamento è riproducibile sulla matrice degli escape time del baseline con l'attraversamento della sezione 5.6.3. Divergenza e spreco sono calcolati anche dal binario CUDA durante lo sweep; cicli e costo dei blocchi sono stati ricavati successivamente dalla stessa matrice, con `mandelbrot/analysis/block_geometry.py`. In entrambi i casi le grandezze dipendono dai soli dati seriali, ed è in questo senso che costituiscono previsioni a priori.

Per ciascuna geometria si considerano quattro grandezze:

- la **divergenza** e lo **spreco medio**, definiti rispettivamente nelle sezioni 5.6.2 e 5.6;
- i **cicli relativi**: poiché un warp termina solo quando termina la sua lane più lenta, i cicli eseguiti con una geometria sono $\sum_{\text{warp}} \ell \cdot \max n_{i,j}$, con ℓ numero di lane del warp; il loro rapporto con quelli della 256×1 è il tempo relativo previsto nell'ipotesi che il tempo sia proporzionale ai cicli;
- il **CoV del costo dei blocchi**, coefficiente di variazione dei cicli eseguiti per blocco, che descrive la granularità con cui il lavoro è suddiviso.

Non si riporta invece il rapporto fra costo massimo e costo medio dei blocchi, che sarebbe l'analogo di λ_p : non appena un blocco cade interamente all'interno dell'insieme, il suo costo è $N_{\max}$ per lane, e il rapporto si riduce a $N_{\max}$ diviso i cicli medi per pixel ($1000/286,4 = 3,49$ per tutte le geometrie orizzontali), senza alcuna informazione sulla forma del blocco.

Quattro partizioni in nove geometrie. Il proxy vale 10 215 per tutte le sei geometrie con `blockDim.x` multiplo di 32. In questo caso, all'interno di ogni warp, `id mod blockDim.x` assume 32 valori consecutivi e `[id/blockDim.x]` resta costante: il warp è formato da 32 pixel contigui della stessa riga, e ogni riga risulta suddivisa in segmenti di 32 pixel indipendentemente da `blockDim.y`. La partizione in warp è quindi la stessa, e con essa divergenza, spreco e cicli; le sei forme differiscono soltanto per la granularità dei blocchi, come mostra la colonna del CoV. Le nove geometrie realizzano pertanto quattro partizioni distinte. Le due compatte, 8×32 e 16×16 ,

Blocco	divergenza	spreco medio	cicli relativi	CoV blocchi	occ.
128×1	10 215	11,24%	1,000	1,41	1,00
256×1	10 215	11,24%	1,000	1,34	1,00
512×1	10 215	11,24%	1,000	1,25	1,00
128×2	10 215	11,24%	1,000	1,41	1,00
64×4	10 215	11,24%	1,000	1,51	1,00
32×8	10 215	11,24%	1,000	1,54	1,00
16×16	5 932	7,82%	0,947	1,59	1,00
8×32	3 689	6,23%	0,928	1,59	1,00
1×256	9 550	11,75%	0,998	1,33	1,00
sparso	131 050	66,31%	2,501	—	—

Tabella 11: Previsioni per le geometrie dello Sweep A, calcolate dalla sola matrice degli escape time del baseline (1024², $N_{\max} = 1000$, kernel *bare*) e, per l’occupancy, dall’API runtime. Le righe sono raggruppate per partizione in warp: orizzontale, compatta, verticale e sparsa. I cicli sono relativi alla 256×1. Il raggruppamento sparso non è realizzabile con una forma di blocco.

riuniscono pixel più vicini nel piano e riducono i cicli del 7,2% e del 5,3%; quella verticale, 1×256, è quasi equivalente all’orizzontale (−0,24%). Fra le forme realizzabili il modello attribuisce dunque alla geometria del blocco un peso di pochi punti percentuali. In media, inoltre, la matrice non conferma che riunire righe diverse aumenti la divergenza: le partizioni compatte la riducono. I casi locali di forte eterogeneità illustrati nella figura 8 restano possibili, ma pesano poco sulla media dell’intera immagine.

Occupancy. L’occupancy prevista vale 1,00 per tutte le geometrie. Il kernel impiega 22 registri per thread, secondo il log di compilazione. Per la compute capability 8.9 un SM dispone di 64 K registri e ospita al più 1 536 thread residenti, cioè 48 warp da 32 thread [9]: i registri basterebbero per circa 90 warp, ben oltre il limite, per cui il vincolo effettivo è il numero di thread e non la disponibilità di registri. Tutte le geometrie usano inoltre blocchi da 128, 256 o 512 thread, divisori dei 1 536 thread di un SM, che viene così riempito senza posti residui. L’occupancy non può dunque distinguere le geometrie, e l’ipotesi della sezione 5.6.2, secondo cui il numero di variabili del kernel FP64 ne avrebbe limitato il valore, è smentita già prima dell’esecuzione.

Il raggruppamento sparso. Nella forma calcolata, il pixel p della matrice linearizzata è assegnato al warp $p \bmod 32\,768$: ogni warp contiene 32 pixel della stessa colonna, a 32 righe di distanza l’uno dall’altro, e campiona quindi l’intera altezza dell’immagine. Il proxy vale 131 050, 12,8 volte quello della partizione orizzontale, lo spreco medio sale al 66% e i cicli a 2,5 volte: è il caso peggiore anticipato nella sezione 5.6.1.

5.6.5 Analisi dei risultati

Sweep A – geometria del blocco. La tabella 12 riporta le nove geometrie, misurate in due campagne distinte; tutte producono lo stesso checksum della baseline seriale. Fra la più veloce (1×256, 5,631 ms) e la più lenta (64×4, 6,091 ms) il divario è dell’8,2%: su questo kernel la geometria del blocco è una leva di second’ordine. I paragrafi seguenti verificano che il divario sia sistematico, lo confrontano con le previsioni della sezione 5.6.4 (occupancy e cicli) ed esaminano infine i due effetti che la tabella delle previsioni non contiene: coalescenza e granularità dei blocchi.

Riproducibilità della classifica. Sei delle nove geometrie hanno `blockDim.x` multiplo di 32 e, come mostrato nella sezione 5.6.4, suddividono l’immagine negli stessi warp: divergenza, spreco e occupancy coincidono, e nessuna di queste grandezze le distingue. Nella campagna a 10

Tabella 12: Sweep A: tempi misurati per le nove geometrie del blocco (1024^2 , $N_{\max} = 1000$, kernel *bare*), nello stesso ordine della tabella 11. Le due colonne di tempo corrispondono ai job 34334 (10 ripetizioni) e 34335 (30 ripetizioni), entrambi su gnode01, partizione *only-one-gpu*. Il tempo relativo è riferito alla 256×1 nella campagna a 30 ripetizioni e va letto contro i cicli relativi previsti; lo speedup è riferito alla baseline seriale del job 34335, $T(1) = 0,720255$ s.

Blocco	$T_{\min}^{(10)}$ [ms]	$T_{\min}^{(30)}$ [ms]	$T/T_{256 \times 1}$	S
128×1	5,802	5,690	0,985	126,6
256×1	5,814	5,776	1,000	124,7
512×1	6,057	5,988	1,037	120,3
128×2	5,984	5,957	1,031	120,9
64×4	6,244	6,091	1,055	118,2
32×8	6,116	6,086	1,054	118,3
16×16	5,877	5,860	1,015	122,9
8×32	5,865	5,752	0,996	125,2
1×256	5,623	5,631	0,975	127,9

ripetizioni i loro tempi minimi coprivano tuttavia un intervallo del 7,6%, più del doppio del vantaggio della 1×256 sulla 256×1 (3,4%). Con una dispersione di tale ampiezza fra configurazioni equivalenti rispetto ai predittori, non era possibile stabilire se la classifica fosse riproducibile. La campagna a 30 ripetizioni risolve la questione: l'ordinamento si ripete con un solo scambio, fra la terza e la quarta posizione (8×32 e 256×1 , distanti lo 0,4%); nessun tempo minimo varia di più del 2,5% fra le due campagne, e quello della 1×256 varia dello 0,14%. Le differenze sono quindi sistematiche e richiedono una spiegazione.

La stessa campagna conferma la scelta di $T_{\min}$ come statistica di riferimento. Per la 8×32 si ha $T_{\text{mean}} = 11,21$ ms a fronte di $T_{\text{median}} = 6,05$ ms: la mediana, allineata a quella delle altre geometrie, esclude un rallentamento dell'intera serie, mentre lo scarto della media corrisponde a un ritardo cumulato di circa 155 ms sulle 30 ripetizioni, compatibile con una singola esecuzione disturbata da un evento esterno. Il minimo non ne risente.

Occupancy. L'occupancy rilevata vale 1,00 per tutte le geometrie, come previsto nella sezione 5.6.4, e non può quindi spiegare le differenze di tempo.

Cicli previsti e tempi misurati. La tabella 11 attribuisce alle partizioni compatte una riduzione dei cicli del 7,2% (8×32) e del 5,3% (16×16) rispetto alla 256×1 . Se il tempo fosse determinato dai cicli dei warp, le due geometrie dovrebbero risultare più veloci in misura analoga. La tabella 12 non lo conferma: la 8×32 guadagna lo 0,4% e la 16×16 è più lenta dell'1,5%.

La previsione non è per questo errata: i cicli sono un calcolo esatto sulla matrice e quantificano correttamente il lavoro mascherato. Ciò che la misura smentisce è l'ipotesi, formulata nella sezione 5.6.2, che tale lavoro sia la leva primaria del tempo di esecuzione: nell'intervallo esplorato, riduzioni dei cicli dei warp fino al 7% non producono guadagni apprezzabili. Individuare la componente che domina il tempo richiederebbe una misura strumentata, non disponibile su questo cluster (si veda oltre).

Coalescenza. Il confronto fra 256×1 e 1×256 era stato progettato per isolare la penalità di coalescenza. Le due geometrie hanno divergenza simile (10 215 e 9 550) e, secondo la tabella 11, un numero di cicli dei warp che differisce dello 0,24%; la prima scrive però 32 indirizzi contigui per warp, la seconda 32 indirizzi distanti 4 096 byte l'uno dall'altro. In presenza di una penalità di coalescenza la 1×256 risulterebbe più lenta; la misura la indica invece più veloce del 2,5% (3,4% nella campagna a 10 ripetizioni).

L'assenza di penalità si spiega confrontando il traffico di memoria con la banda disponibile. Il kernel scrive 4 MiB per lancio in 5,63 ms, pari a circa 0,74 GB/s, a fronte degli 864 GB/s di

banda di memoria della scheda [8]. Anche ammettendo che la perdita di coalescenza moltiplichi il traffico per il fattore massimo di 32 (sezione 5.6), si otterrebbero circa 24 GB/s, meno del 3% della banda. In nessuna delle due configurazioni il canale di memoria si avvicina alla saturazione, e il numero di transazioni non incide quindi sul tempo. Il risultato costituisce la verifica sperimentale di quanto la sezione 5.6.2 aveva argomentato per via teorica: il kernel è limitato dal calcolo e non dalla banda.

Granularità dei blocchi. Fra le tre geometrie orizzontali pure, che condividono partizione in warp, divergenza, spreco e occupancy, il tempo cresce con la dimensione del blocco in entrambe le campagne: 128×1 (8192 blocchi da 4 warp, 5,690 ms), 256×1 (4096 blocchi da 8 warp, 5,776 ms), 512×1 (2048 blocchi da 16 warp, 5,988 ms). Poiché l'unica grandezza che varia è il numero di blocchi, la differenza del 5% va attribuita alla granularità con cui il lavoro viene distribuito sui 142 SM del chip [8]. L'interpretazione coerente con i dati è che blocchi più piccoli consentano di ripartire il carico fra gli SM in modo più uniforme, avvicinando il completamento dell'ultimo SM a quello medio. Ciò vale nonostante i blocchi più piccoli abbiano, singolarmente, un costo più variabile (CoV 1,41 contro 1,25): è il loro numero maggiore per SM a mediare il carico. Si tratta dello stesso meccanismo osservato con il chunk in OpenMP (sezione 5.3.4), dove a 32 thread `dynamic`, 1 risultava cinque volte più veloce di `dynamic`, 64. L'effetto è qui molto più contenuto perché anche la geometria più grossolana, con 2048 blocchi, resta di gran lunga più fine dei 16 chunk di `dynamic`, 64.

La granularità non spiega tuttavia tutte le differenze osservate. Le geometrie 128×2 , 64×4 e 32×8 hanno la stessa partizione in warp e lo stesso numero di thread per blocco della 256×1 , dunque la stessa granularità, eppure risultano più lente dal 3 al 5,5%. Questo scarto e il primato della 1×256 sono i due risultati dello sweep che nessuna delle grandezze previste spiega, e vengono pertanto riportati come questioni aperte.

Assenza di una misura strumentata. In entrambi i job la misura di conferma con `ncu` è fallita con l'errore `ERR_NVGPUCTRPERM`: l'accesso ai performance counter della GPU è riservato agli amministratori, secondo l'impostazione predefinita di NVIDIA per i sistemi multiutente. L'esecuzione è proseguita, poiché la chiamata era protetta in previsione di questo caso. Divergenza e occupancy restano quindi prive di una conferma strumentata, e il proxy deterministico è l'unica misura disponibile della divergenza, come previsto nella sezione 5.6.2. Per la stessa ragione le due questioni aperte dello Sweep A non possono essere indagate su questo cluster.

Il raggruppamento ciclico sulla GPU. Il raggruppamento sparso, analogo GPU della decomposizione ciclica, richiederebbe secondo la tabella 11 2,5 volte i cicli della partizione orizzontale, con uno spreco medio del 66%. Su MPI la stessa idea — mescolare righe leggere e pesanti — era il rimedio allo sbilanciamento di carico (sezione 5.4.4); sotto SIMT produce l'effetto opposto, perché riunisce nello stesso warp pixel con escape time eterogenei e moltiplica i cicli mascherati. La causa è unica, la varianza per-pixel degli escape time, ma i rimedi sono contrari: su memoria distribuita conviene mescolare il carico fra le unità di calcolo, sotto SIMT conviene raggruppare pixel simili.

La portata di questa conclusione va però delimitata. Il raggruppamento sparso non è realizzabile con una forma di blocco, perché richiederebbe una permutazione degli indici nel kernel, e non è stato misurato. Lo Sweep A ha inoltre mostrato che, nell'intervallo esplorato, il tempo è poco sensibile ai cicli dei warp: una riduzione del 7,2% si è tradotta in un guadagno dello 0,4%. Che un aumento dei cicli di 2,5 volte produca un rallentamento comparabile resta dunque una previsione del modello, non un risultato sperimentale.

Throughput e confronto con la CPU. La geometria più veloce raggiunge 369 GFLOP/s. La documentazione NVIDIA riporta per la L40, basata sullo stesso chip AD102 della L40S, un throughput FP32 di picco di 90,5 TFLOPS, e un rapporto di 64 fra throughput FP32 e FP64 [8, 9]: il picco in doppia precisione è quindi di circa 1 414 GFLOP/s. Il kernel se ne colloca al 26%, mentre rispetto al picco FP32 si colloca allo 0,4%. Le due percentuali descrivono aspetti distinti: la prima indica quanto siano sfruttate le unità di calcolo in doppia precisione, la seconda quanto sia sfruttata la scheda nel suo complesso. Il divario dipende dalla doppia precisione, imposta dal contratto di riproducibilità, e motiva un eventuale esperimento in singola precisione; i picchi sono valori nominali, non misurati sul cluster.

Lo speedup rispetto alla baseline seriale misurata nello stesso job è $S = 128$. Il confronto più significativo è però con il miglior risultato CPU della campagna, il master-worker MPI con 32 unità di calcolo (23,156 ms, sezione 5.4.4): una sola GPU è 4,11 volte più veloce di 32 core. Il trasferimento device→host richiede fra 0,214 e 0,228 ms, circa il 3,8% di $T_{\min}$ per la geometria più veloce, e non varia apprezzabilmente fra le geometrie, come atteso dato che la quantità di dati copiati è la stessa.

Sweep B – saturazione e trasferimento. La tabella 13 riporta la curva di saturazione (B1) e la frazione di trasferimento al variare di $N_{\max}$ (B2). Tutte le sei configurazioni riproducono il checksum della rispettiva baseline seriale, misurata nello stesso job.

Tabella 13: Sweep B: saturazione (B1, $N_{\max} = 1000$) e frazione di trasferimento (B2, risoluzione 2048²), con blocco 256×1; job 34424 su gnode01, 30 ripetizioni per la GPU e 3 per il seriale. Π è il throughput effettivo in GFLOP/s e S lo speedup rispetto al seriale della stessa configurazione; copia/ T è la frazione transfer_time/ $T_{\min}$; la banda è quella effettiva della copia device→host, cioè i byte copiati divisi per transfer_time. Il punto 2048², $N_{\max} = 1000$ è comune ai due esperimenti.

Sweep	problema	$T_{\min}$ [ms]	Π [GFLOP/s]	S	copia/ T [%]	banda [GB/s]
B1	512 ²	2,565	202,5	70,4	3,49	11,7
B1	1024 ²	5,735	362,2	125,8	3,91	18,7
B1	2048 ²	19,131	434,3	150,6	4,57	19,2
B1	4096 ²	75,705	439,0	152,2	3,74	23,7
B2	$N_{\max}=500$	10,446	406,9	140,0	7,08	22,7
B2	$N_{\max}=1000$	19,131	434,3	150,6	4,57	19,2
B2	$N_{\max}=5000$	87,474	465,2	162,5	0,84	23,0

Controllo fra i due job. Il punto a 1024² e $N_{\max} = 1000$ ripete la configurazione 256×1 dello Sweep A: 5,735 ms contro 5,776 ms, uno scarto dello 0,7%, entro la variabilità osservata fra job distinti. Il confronto vale più di quanto sembri, perché i log mostrano che i due job hanno usato due schede L40S fisicamente diverse: la differenza fra le due schede non supera il rumore di misura, e le due campagne sono direttamente confrontabili. Anche la baseline seriale coincide entro lo 0,2%.

Saturazione (B1). Il throughput effettivo cresce con la risoluzione e poi si stabilizza (figura 10): 202, 362, 434 e 439 GFLOP/s da 512² a 4096². Fra le ultime due taglie il guadagno è dell'1% a fronte di un lavoro quadruplicato: la scheda è satura da circa 2048², cioè da circa quattro milioni di pixel. Poiché il costo medio per pixel è identico in tutte le taglie (248 iterazioni), la differenza dipende soltanto da quanto efficacemente la GPU viene sfruttata. Lo speedup rispetto a un core segue lo stesso andamento, da 70 a 152, mentre il seriale resta fermo a circa 2,88 GFLOP/s.

Dei tre fattori attesi, il trasferimento si può escludere: pesa fra il 3,5 e il 4,6% del tempo a ogni taglia, e non può spiegare un throughput dimezzato. Anche un costo fisso di lancio spiega poco. I tempi delle due taglie maggiori crescono in proporzione ai pixel, e la loro retta attribuisce al costo indipendente dalla taglia circa 0,3 ms; a 512², però, il tempo misurato supera quella

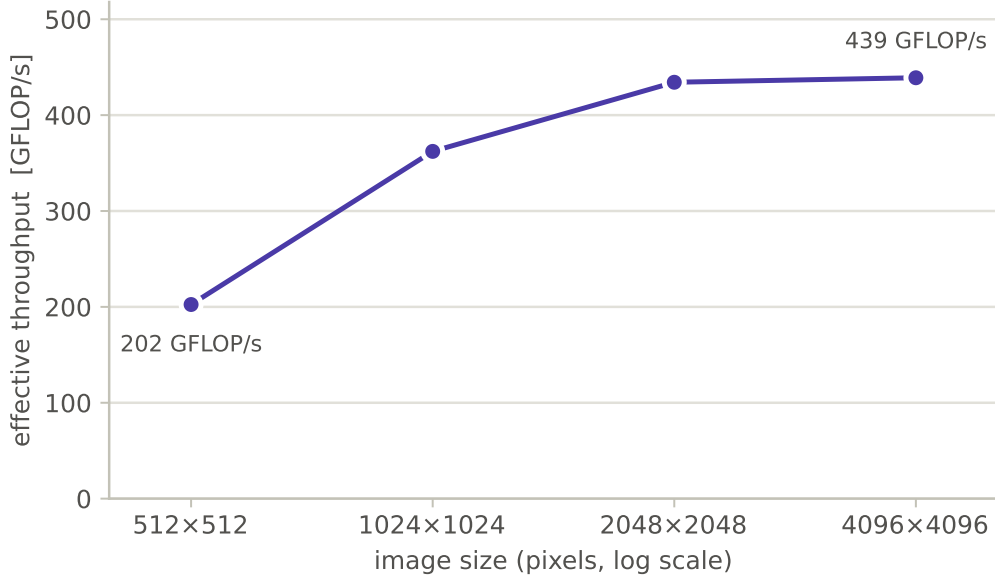


Figura 10: Throughput effettivo della GPU in funzione della dimensione dell'immagine (Sweep B1: blocco 256×1 , $N_{\max} = 1000$, job 34424). Il throughput si stabilizza da 2048^2 ; il seriale, a circa 2,88 GFLOP/s, non è rappresentabile sulla stessa scala.

previsione del 77%, e a 1024^2 del 15%. La penalità delle taglie piccole è quindi compatibile con il terzo fattore, il sottoutilizzo della scheda: con pochi pixel non ci sono abbastanza warp da tenere occupati tutti gli SM. Senza performance counter non è possibile verificarlo direttamente. Il proxy di divergenza diminuisce con la risoluzione, da 18 012 a 3 154, perché pixel adiacenti diventano più simili; lo Sweep A ha però mostrato che i cicli sprecati incidono poco sul tempo, e non possono spiegare la saturazione.

Rispetto al seriale la GPU conviene già alla taglia minima ($S = 70$). Il confronto con la CPU parallela è disponibile solo a 1024^2 , dove una GPU resta 4,04 volte più veloce del master-worker MPI su 32 unità, in accordo con lo Sweep A.

Trasferimento (B2) e posizionamento NUMA. A risoluzione fissa i byte copiati sono costanti (16 MiB), mentre il lavoro cresce con $N_{\max}$: la frazione del tempo spesa nella copia scende dal 7,1% a $N_{\max} = 500$ allo 0,8% a $N_{\max} = 5000$, come atteso. Un modello lineare sui tre punti, $T = c + \tau \mathcal{W}$, riproduce i tempi entro 0,06 ms e separa un costo di circa 1,5 ms indipendente da $N_{\max}$ (copia, allocazione e lancio a questa taglia) da un throughput marginale di 473 GFLOP/s, il valore verso cui tende quello effettivo al crescere del lavoro.

Il log del job documenta il posizionamento: la GPU assegnata ha come core locali 0–31, cioè il nodo NUMA 0, mentre il processo è stato eseguito sul core 42, sul socket 1. È il caso cross-NUMA descritto nella sezione 5.6.2: ogni copia attraversa anche l'interconnessione fra i socket. La banda effettiva cresce con la dimensione della copia (figura 11), da 11,7 GB/s per 1 MiB a 23,7 GB/s per 64 MiB. Interpolando il tempo di copia in funzione dei byte si ottengono una banda di circa 24 GB/s e un costo fisso di circa 0,07 ms per copia, che penalizza le copie piccole. La banda asintotica è quindi circa il 77% dei 31,5 GB/s nominali del PCIe 4.0 x16. A parità di byte la banda oscilla fra 19,2 e 23,0 GB/s nelle tre esecuzioni a 2048^2 : la singola copia è soggetta a una variabilità non trascurabile.

Quanto della riduzione rispetto al nominale sia dovuto al passaggio fra i socket non si può stabilire con un solo job. Il tempo misurato somma i tre contributi, e separarli richiederebbe un confronto con il caso locale, che SLURM non permette di scegliere; il valore nominale, inoltre, si riferisce al solo collegamento e non comprende l'overhead di protocollo. Lo Sweep A, eseguito

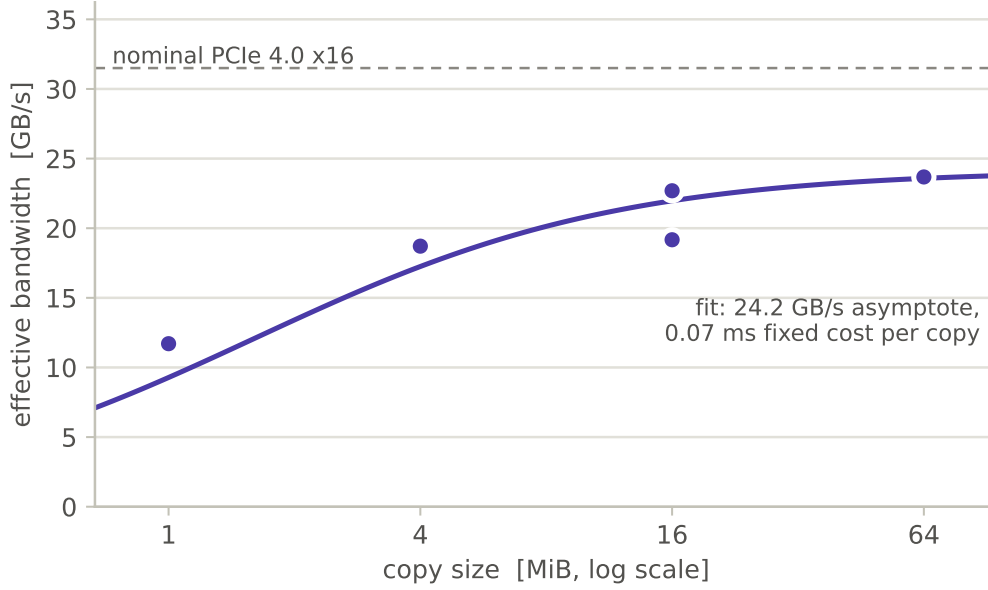


Figura 11: Banda effettiva della copia device→host in funzione della dimensione della copia, per le sei configurazioni dello Sweep B (job 34424, processo e GPU su socket diversi). La curva è il modello lineare del tempo di copia, con banda asintotica di 24,2 GB/s e costo fisso di 0,07 ms per copia; la linea tratteggiata è la banda nominale del PCIe 4.0 x16.

senza il log NUMA, ha copiato 4 MiB in 0,217 ms contro i 0,224 ms di questo job: un valore compatibile, che non permette però di attribuirgli un posizionamento.

Sintesi. Lo studio della GPU si chiude su tre risultati. La forma del blocco è una leva di second'ordine, e le grandezze calcolabili dalla matrice degli escape time, pur esatte, non governano il tempo. La scheda si satura da circa quattro milioni di pixel, dove raggiunge 439 GFLOP/s e uno speedup di 152 rispetto a un core; sotto quella taglia il calo non dipende dal trasferimento ed è compatibile con il sottoutilizzo della scheda. Il trasferimento, infine, è un costo marginale che si ammortizza con il lavoro, ma la sua banda dipende da un posizionamento che SLURM non garantisce: nel job misurato la copia attraversava i due socket e raggiungeva circa tre quarti della banda nominale.

6 Conclusioni

L'elaborato si era proposto di confrontare OpenMP, MPI e CUDA sullo stesso problema, misurandone il comportamento con metriche consolidate e mettendo in relazione i risultati con i modelli analitici del calcolo parallelo. La tesi formulata nell'introduzione era che un'unica causa, la variabilità del numero di iterazioni per pixel, si manifestasse in forme diverse a seconda dell'architettura: come sbilanciamento del carico fra thread e processi sulla CPU, come divergenza dei warp sulla GPU. Le sezioni che seguono riassumono che cosa le misure hanno confermato, che cosa hanno smentito e che cosa è emerso sul problema.

6.1 Il problema: un'irregolarità misurabile prima di parallelizzare

La baseline seriale ha mostrato che il costo per iterazione è costante entro l'1,3% su tutte le configurazioni misurate: il tempo seriale è proporzionale al numero totale di iterazioni $\mathcal{W}$, che si può quindi usare al posto del tempo per ragionare sul lavoro. Il profilo di carico per riga è fortemente irregolare ($\lambda \approx 2,76$, $\text{CoV} \approx 0,97$), e questa irregolarità dipende dalla geometria dell'insieme più che dalla soglia iterativa: portando $N_{\max}$ da 500 a 5000, λ cresce del 2,9%, mentre il lavoro cresce di un fattore 9,57 e solo lo 0,116% dei pixel cambia classificazione.

La conseguenza metodologica è il risultato più generale del lavoro: il profilo di carico e la matrice degli escape time, ottenuti da una sola esecuzione seriale, bastano a formulare previsioni quantitative sulle versioni parallele prima di scriverle. Sulla CPU queste previsioni si sono rivelate accurate; sulla GPU descrivono correttamente il lavoro, ma non il tempo.

6.2 CPU: la regola di assegnamento decide lo speedup

Sulla CPU le previsioni ricavate dal solo profilo seriale sono state confermate (sezione 5.5): 19 dei 25 punti di misura rientrano nell'1% del limite predetto. I sei restanti, tutti a $p \geq 16$, se ne discostano fra il 2,6 e l'8,5%, per costi che il modello non include. La regola a blocchi, realizzata con `schedule(static)`, raggiunge il proprio limite entro lo 0,6%: la perdita del 62% di efficienza a 32 thread è dovuta quasi per intero alla contiguità delle righe costose, che finiscono nello stesso blocco. Cambiando soltanto la regola, senza aggiungere comunicazione, la decomposizione ciclica resta entro l'1% dall'ideale fino a 8 processi; a 16 e 32 se ne allontana, per una causa ancora da verificare. Le regole dinamiche a grana fine arrivano a $S = 30,99$ (OpenMP `dynamic,1`) e $S = 31,10$ (MPI master-worker) su 32 unità. Nello sweep MPI il rapporto fra lo speedup peggiore (block) e il migliore (master-worker) è 2,66, pari allo sbilanciamento λ_{32} previsto per i blocchi.

Il dato che accomuna i due paradigmi è che la previsione dipende dalla *regola* e non dal meccanismo che la realizza: una clausola `schedule(static)` in memoria condivisa e una decomposizione esplicita con gather in memoria distribuita cadono sullo stesso limite entro l'1% fino a $p = 16$. Anche il confronto fra i due scheduler dinamici lo conferma: fino a $p = 16$ il passaggio da memoria condivisa a scambio di messaggi costa meno del 2%.

6.3 GPU: la stessa irregolarità pesa poco sul tempo

Una sola L40S calcola l'immagine di riferimento 128 volte più velocemente di un core e 4,11 volte più velocemente del miglior risultato CPU, il master-worker MPI su 32 unità. Tutte le geometrie del blocco riproducono il checksum della baseline, e la geometria stessa si è rivelata una leva di second'ordine: fra la più veloce e la più lenta corre l'8,2%.

Le grandezze calcolabili dalla matrice degli escape time sono esatte, ma spiegano poco delle differenze di tempo. L'occupancy vale 1 per tutte le geometrie, perché il kernel non esaurisce i registri. La perdita di coalescenza non comporta una penalità misurabile: la 1×256 , che la subisce, è la geometria più veloce, e il traffico di memoria resta sotto il 3% della banda anche nel caso peggiore. Una riduzione prevista del 7,2% dei cicli eseguiti si traduce in un guadagno misurato dello 0,4%. L'unica differenza riconducibile alla distribuzione del carico è quella fra le geometrie orizzontali pure, circa il 5%, interpretata come effetto della granularità con cui il lavoro si ripartisce fra gli SM. L'irregolarità del carico, che sulla CPU spiega quasi per intero le prestazioni, sulla GPU ne spiega quindi soltanto pochi punti percentuali.

Lo Sweep B aggiunge tre elementi. La scheda si satura da circa 2048^2 pixel, dove il throughput si stabilizza attorno a 440 GFLOP/s e lo speedup rispetto a un core a 152. Alle taglie inferiori il rendimento cala, e né il trasferimento né un costo fisso di lancio bastano a spiegarlo: il calo è compatibile con il sottoutilizzo degli SM, che senza performance counter non è stato possibile verificare. La copia dei risultati pesa fra lo 0,8 e il 7,1% del tempo e si ammortizza al crescere di $N_{\max}$. Infine, la posizione del processo rispetto alla GPU non è garantita: nel job misurato la copia attraversava i due socket e raggiungeva una banda asintotica di circa 24 GB/s, tre quarti di quella nominale del PCIe, senza che fosse possibile isolare il contributo del passaggio fra i socket.

6.4 Le ipotesi iniziali alla prova dei dati

La tesi dell'unica causa. È confermata come principio che organizza l'intero lavoro, ma con un peso molto diverso nei due casi. Sulla CPU la variabilità degli escape time determina lo speedup, e lo fa in modo prevedibile: le regole statiche saturano il limite predetto entro lo 0,6%, e gli scostamenti maggiori corrispondono a costi esterni al modello. Sulla GPU la stessa variabilità è misurabile come divergenza dei warp, ma il suo effetto sul tempo è marginale nell'intervallo esplorato. Anche la conclusione più suggestiva, che la stessa causa richieda rimedi opposti — mescolare il carico sulla CPU, raggruppare pixel simili sulla GPU — resta per la GPU una previsione del modello: il raggruppamento sparso non è realizzabile con una forma di blocco e non è stato misurato, e lo Sweep A indica che il tempo è poco sensibile ai cicli dei warp.

Amdahl e Karp–Flatt. La legge di Amdahl, con una frazione seriale trascurabile, prevede uno speedup quasi ideale e da sola non spiega le perdite osservate. La metrica di Karp–Flatt le segnala correttamente, ma non misura una frazione seriale: per la regola a blocchi $e(p)$ decresce con p e segue la forma $(\lambda_p - 1)/(p - 1)$, cioè lo sbilanciamento riscritto in un'altra variabile. È diagnostico il suo andamento, più che il suo valore: una $e(p)$ decrescente indica uno sbilanciamento il cui limite cresce con p , una crescente un limite fisso, come la granularità o la contesa sulla coda.

Andamenti più che valori assoluti. L'introduzione aveva anticipato che i valori assoluti, legati alle risorse disponibili, contassero meno degli andamenti. I risultati lo confermano. Le previsioni sulla CPU sono rapporti, e non dipendono dal valore di T_1 . Le sei baseline seriali dei diversi job concordano entro lo 0,8%, e due schede L40S fisicamente diverse hanno dato tempi entro lo 0,7%: nessuna di queste differenze modifica le conclusioni.

6.5 Risultati inattesi

Alcuni risultati non erano previsti, e sono fra i più istruttivi.

- **La granularità può annullare il bilanciamento dinamico.** `dynamic,64` si ferma a $S = 6,17$ già da 8 thread: con soli 16 chunk il tempo non può scendere sotto quello del chunk più pesante, e lo scheduling dinamico eredita il limite di una decomposizione statica a 16 blocchi.
- **Una politica adattiva può lavorare contro il problema.** `guided` perde circa il 31% di efficienza a ogni grado di parallelismo, perché assegna blocchi contigui grandi proprio all'inizio, quando il carico andrebbe compensato.
- **Il modello di memoria pesa poco.** A 32 unità il master-worker MPI è più veloce dello scheduler dinamico di OpenMP dell'1,1%; la contesa sulla coda condivisa di OpenMP è una spiegazione plausibile, non verificata. A parità di core allocati, però, OpenMP torna avanti di circa il 2%.
- **Uno speedup superlineare solo apparente.** A $p = 2$ tutte le politiche OpenMP superano di poco lo speedup ideale ($S \approx 2,011$). La causa è la variabilità di T_1 fra job distinti: la baseline di un altro job, più veloce di 3,3 ms, riporta lo speedup a 2,002. Per questo ogni misura è riferita alla baseline del proprio job, misurata sul nodo nello stesso stato.
- **Sulla GPU le leve attese spiegano poco.** Occupancy, coalescenza e divergenza, individuate in fase di progettazione come fattori determinanti, non spiegano le differenze di tempo fra le geometrie. Restano aperti il primato della geometria 1×256 e il ritardo delle

forme bidimensionali coalescenti, che senza accesso ai performance counter non è stato possibile indagare.

6.6 Limiti e sviluppi

I risultati vanno letti alla luce dell'ambiente di esecuzione (sezione 1.1). Tutte le misure provengono da un unico nodo: MPI è stato eseguito su memoria condivisa, e la frazione di comunicazione misurata, mai superiore al 3,4%, è un limite inferiore ottimistico rispetto a un'esecuzione distribuita, nella quale il master-worker potrebbe perdere il vantaggio sul ciclico. Il nodo è condiviso con altri utenti, e il minimo sulle ripetizioni ne ha assorbito il rumore; anche la posizione del processo rispetto alla GPU dipende dall'assegnazione di SLURM e non è controllabile. Sulla GPU tutte le misure usano una sola scheda. La doppia precisione, imposta dalla riproducibilità, colloca il kernel su un picco nominale 64 volte inferiore a quello FP32; i picchi, inoltre, sono documentati per la L40 e non misurati. L'impossibilità di accedere ai performance counter impedisce infine di spiegare i risultati residui.

Restano come sviluppi naturali il Buddhabrot, proposto nell'introduzione per osservare il regime dominato dalla contesa sulla memoria condivisa, e non affrontato; le versioni ibride, che combinano MPI con OpenMP o con CUDA; un esperimento in singola precisione, per quantificare il compromesso fra precisione e throughput; la misura diretta del costo di riordino nella decomposizione ciclica MPI, insieme alla rimisura del punto a $p = 16$; l'accesso ai performance counter, per indagare le due questioni aperte dello Sweep A; e un'esecuzione su più nodi, che darebbe alla frazione di comunicazione il suo valore reale.

Bibliografia

- [1] fractalsfoundation.org. *What are fractals?* <https://fractalfoundation.org/resources/what-are-fractals/>. Accessed on July, 4th, 2026 (cit. a p. 3).
- [2] Juan Carlos Ponce Campuzano. *The Mandelbrot set*. https://complex-analysis.com/content/mandelbrot_set.html. Accessed on September, 3rd, 2026 (cit. alle pp. 3, 4).
- [3] matematica.unibocconi.eu. *Mandelbrot e la geometria frattale*. <https://matematica.unibocconi.eu/articoli/mandelbrot-e-la-geometria-frattale>. Accessed on September, 3rd, 2026 (cit. a p. 3).
- [4] jlmarguz. *Navigating the Mandelbrot Set*. <https://fractals.marguz.net/2025/04/17/navigating-the-mandelbrot-set/>. Quaint Symmetries, 17 aprile 2025. Accessed on September, 15th, 2026 (cit. a p. 5).
- [5] Robert L. Devaney. *The Geometry of the Mandelbrot Set: Antennas, Spokes, and Periods*. <https://math.bu.edu/DYSYS/FACGEOM/node3.html>. Accessed on September, 3rd, 2026 (cit. a p. 5).
- [6] Jos Leys. *Alla ricerca dell'insieme di Mandelbrot in tre dimensioni*. <https://maddmaths.simai.eu/divulgazione/alla-ricerca-dellinsieme-di-mandelbrot-in-tre-dimensioni/>. Accessed on September, 3rd, 2026 (cit. alle pp. 5, 6).
- [7] Mitsuhiro Shishikura. «The Hausdorff Dimension of the Boundary of the Mandelbrot Set and Julia Sets». In: *Annals of Mathematics* 147.2 (1998), pp. 225–267. DOI: 10.2307/121009 (cit. a p. 17).
- [8] NVIDIA Corporation. *NVIDIA Ada GPU Architecture*. <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>. Accessed on September, 14th, 2026 (cit. alle pp. 36, 37, 45, 46).
- [9] NVIDIA Corporation. *CUDA Programming Guide: Compute Capabilities*. <https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html>. Accessed on September, 14th, 2026 (cit. alle pp. 36, 43, 46).